The Chess Algorithm: A Novel Metaheuristic Optimization Technique with Applications in Transportation Network Engineering

Seyed Saber Naseralavi Seyedali Mirjalili

2026-07-04

We introduce the Chess Algorithm (CA), a population-based metaheuristic that borrows its search logic from the strategy of chess rather than from foraging or hunting behavior in nature. The population is not homogeneous: at every iteration agents are ranked and assigned the roles of King, Queens, Rooks, Bishops, Knights, and Pawns, and each role moves according to an operator abstracted from the geometry of the corresponding piece. Strategic ideas from the game supply the control machinery, and a lightweight state machine reads the divergence, initiative, and local-search success of the population each iteration to decide which of them apply: development, sacrifice, pinning, castling, en passant, and threefold repetition throughout; Novotny interference, the knight's fork, the royal council, and a blockade response combining a King's march with a Tal-style speculative sacrifice in the tactically richer phases of the search. Every mechanism is mapped explicitly onto its established operator family — success-rule step adaptation, differential mutation, arithmetic recombination, and related traditions — directly engaging the standard criticism that metaphor-driven algorithms repackage known operators under new vocabulary. The comparison roster reflects this: alongside GA, PSO, SA, GWO, and WOA, it includes two competition-grade optimizers, L-SHADE and CMA-ES, so CA's standing is measured against the strongest representatives of the operator families it draws on, not only against other metaphor-derived methods. On six standard 30-dimensional benchmarks against GA, PSO, SA, and GWO under matched evaluation budgets, CA is significantly better than PSO and SA on every function and than GA on five of six (tied on the sixth), while GWO retains its established advantage on this classical suite. On the full CEC-2017 benchmark suite — all 29 usable functions, validated through a two-library data-integrity audit that identified six defective implementations in the primary third-party port and restored all six from an independent, official-data source, $D = 30$, 30 independent runs — CA achieves the best mean Friedman rank among the six classical/moderate baselines (CA, CA-static, GWO, PSO, GA, WOA), with zero losses to WOA, one to GWO, and one to PSO, while L-SHADE and CMA-ES rank first and second overall and beat CA on the large majority of functions, reported without qualification. The same ordering — L-SHADE and CMA-ES first, CA best of the remainder — holds on the CEC-2022 suite at both official dimensionalities (16 functions validated after an analogous audit) and on seven classic constrained engineering design problems, where CA loses only once to the five classical/moderate baselines across 35 comparisons but loses to both specialized methods on every problem. A granular ablation isolating each of CA's twelve mechanisms explains this ordering rather

than merely describing it: two mechanisms — en passant and the knight's fork — carry the large majority of CA's performance, and they are precisely the success-rule step-adaptation and differential-mutation families that CMA-ES and L-SHADE embody in fully adaptive form; a one-at-a-time parameter sensitivity analysis and a measured computational-cost comparison (evaluation overhead 12.4%, replacing an earlier estimate) complete the internal audit. Practical value is assessed on transportation problems: on the coordinated signal timing of an eight-intersection arterial with a Webster/HCM delay model, CA is significantly better than GA and GWO and loses narrowly to L-SHADE and CMA-ES; on a continuous berth allocation instance with a known global optimum, an extended comparison against six algorithms from the independent mealpy library plus L-SHADE and CMA-ES shows CA ahead of all six mealpy baselines and behind the two specialized methods. Throughout, an ablation configuration with the adaptive control layer disabled (CA-static) is carried alongside CA to isolate what that layer contributes. The results support a measured claim, deliberately narrower than a performance claim: CA is a transparent, mathematically specified, and fully reproducible architecture that is competitive with classical and widely used metaheuristics and does not match specialized competition-grade optimizers — and its ablation identifies exactly which of its components explain both facts.

[1] *Seyed Saber Naseralavi (corresponding author,* `saber_naseralavi@uk.ac.ir`*) — Department of Civil Engineering, Faculty of Engineering, Shahid Bahonar University of Kerman, Kerman, Iran.*
[2] *Seyedali Mirjalili — Centre for Artificial Intelligence Research and Optimization, Torrens University Australia, Brisbane, Australia.*
An HTML version of this article, together with all code and data, is available at https://sabernaseralavi-60.github.io/2026_Chess-Algorithm/.

Author note. Dr. Seyedali Mirjalili is listed as an invited co-author; his participation is pending confirmation. The invitation reflects how much this study owes to his work on swarm and nature-inspired optimization, and his name will be retained only with his explicit consent.

Keywords: metaheuristics; chess algorithm; adaptive optimization; global optimization; CEC-2017 benchmarks; constrained engineering design; traffic signal timing; berth allocation; transportation networks.

1 Introduction

1.1 Motivation

A large share of the decisions a transportation engineer has to make can be written, after enough simplification, as a global optimization problem:

$$\min_{\mathbf{x}\in\Omega} f(\mathbf{x}), \qquad \Omega = \{\mathbf{x} \in \mathbb{R}^D : l_j \le x_j \le u_j,\ j = 1, \dots, D\}. \tag{1}$$

The trouble is what f looks like in practice. Signal timing, network design, transit scheduling, detector placement — in each of these the objective is non-convex, often non-differentiable, riddled with local optima, and sometimes expensive to evaluate (Ceylan & Bell, 2004; Osorio

& Bierlaire, 2013). Exact methods rarely scale to instances of realistic size. This is why metaheuristics have retained their prominence for three decades: they trade the guarantee of optimality for robustness, generality, and a computationally tractable budget.

1.2 A brief map of the metaheuristic landscape

The field grew from two roots. Evolutionary computation gave us the Genetic Algorithm (Goldberg, 1989; Holland, 1975) and Evolution Strategies (Beyer & Schwefel, 2002). Trajectory methods gave us Simulated Annealing (Kirkpatrick et al., 1983), whose central idea — accept a worse solution now and then, so you can escape a local minimum — turned out to be one of the most durable in the field. Swarm intelligence arrived in the 1990s with Particle Swarm Optimization (Kennedy & Eberhart, 1995), Ant Colony Optimization (Dorigo et al., 1996), Differential Evolution (Storn & Price, 1997), and later the Artificial Bee Colony (Karaboga & Basturk, 2007). More recently, Mirjalili and colleagues built a productive line of algorithms by abstracting the social and hunting behavior of animals into compact operators: the Grey Wolf Optimizer (Mirjalili et al., 2014), the Ant Lion Optimizer (Mirjalili, 2015b), the Dragonfly Algorithm (Mirjalili, 2016a), the Whale Optimization Algorithm (Mirjalili & Lewis, 2016), the Sine Cosine Algorithm (Mirjalili, 2016b), and the Salp Swarm Algorithm (Mirjalili et al., 2017), among others.

This literature reveals two notable patterns. First, almost every swarm algorithm treats its agents identically: a single update equation is applied to all agents, with only position and fitness distinguishing one from another. However, division of labor among *unequal* agents is a well-established principle in both biological systems and human strategic behavior. Second, the performance of a metaheuristic is largely determined by how it schedules the transition from exploration to exploitation (Mirjalili et al., 2014), and mechanisms that adapt this transition based on observed search dynamics — rather than on elapsed time alone — remain uncommon in the literature.

The justification for proposing an additional metaheuristic should also be stated explicitly. The No-Free-Lunch theorem (Wolpert & Macready, 1997) precludes the existence of a universally dominant optimizer; accordingly, the objective of this work is not universal superiority, but rather to contribute a substantively different inductive bias and to identify the problem class for which that bias is advantageous.

A third consideration is critical rather than structural. The rapid proliferation of metaphor-named algorithms has drawn severe and largely justified criticism (Sörensen, 2015), on the grounds that novel vocabulary frequently repackages established operators without advancing the underlying science. This paper accepts the substance of that criticism and structures its response into the study itself: Section 2.12 maps every mechanism of the proposed algorithm onto its established operator family with the corresponding citation, Section 7.1 measures each mechanism's isolated contribution empirically, and the comparison roster includes two competition-grade optimizers — L-SHADE (Tanabe & Fukunaga, 2014) and CMA-ES (Hansen & Ostermeier, 2001) — precisely so that the algorithm's standing is assessed against the strongest representatives of the operator families it draws on, not only against other metaphor-derived methods.

1.3 Why chess?

Chess offers precisely the two properties identified as absent above. Its pieces are radically unequal: the queen sweeps the whole board, rooks travel along files and ranks, bishops along

diagonals, knights jump over anything in the way, and pawns crawl forward one square at a time while quietly defining the structure of the position. Mapped onto a search population, this suggests sub-groups performing qualitatively different moves around one common reference point — the King, which in our setting is the best solution found so far.

The game also comes with a mature strategic vocabulary, refined over centuries of analysis. Development says: mobilize your forces early, consolidate later. A sacrifice gives up material now for a positional advantage later. Pinning immobilizes a piece against something more valuable behind it. Castling is a safeguarded structural rearrangement; en passant, an opportunistic capture available only under narrow conditions; the threefold-repetition rule declares a draw when the position keeps repeating. Each of these has a natural algorithmic reading — exploration schedules, acceptance of deteriorating moves, variable fixation, elite restructuring, adaptive local search, and diversity preservation, respectively.

The Chess Algorithm (CA) proposed here formalizes that mapping. To preempt a common misunderstanding: CA does not play chess and does not search a game tree. It borrows the strategic grammar of the game exclusively, and its per-iteration computational cost is comparable to that of GA or PSO.

1.4 Contributions and scope

The paper contributes, in order: a complete mathematical specification of CA — initialization, rank-based role assignment, movement operators, strategic mechanisms, and the adaptive control layer that governs when each applies, including a state machine that reads population divergence, initiative, and local-search success each iteration and switches between opening, middlegame, closed-position, and endgame tactical repertoires, with several operators new to this paper (Novotny interference, the knight's fork, the royal council, a King's-march blockade response, and a Tal-style speculative-sacrifice reheat), together with an explicit mapping of every mechanism onto its established operator family (Section 2); a controlled benchmark study on six standard test functions in 30 dimensions against GA, PSO, SA, and GWO under matched budgets, with nonparametric testing following Derrac et al. (2011) (Section 3); a full evaluation on the CEC-2017 benchmark suite against an eight-algorithm roster that includes the competition-grade baselines L-SHADE (Tanabe & Fukunaga, 2014) and CMA-ES (Hansen & Ostermeier, 2001) alongside GWO, PSO, GA, and WOA, including a two-library data-integrity audit that identified six defective function implementations in the third-party `opfunu` port and restored all six from an independent implementation carrying the official reference data (Section 4); an evaluation on the CEC-2022 suite at both of its official dimensionalities under the same roster (Section 5); an evaluation on seven classic constrained engineering design problems (Section 6); a granular per-mechanism ablation study, a one-at-a-time parameter sensitivity analysis, and a measured computational-cost comparison with exact evaluation counts (Section 7); a transportation case study on coordinated signal timing of an eight-intersection arterial with a Webster/HCM delay model (Roess et al., 2019; Webster, 1958) (Section 8); an extended comparison against six third-party implementations from the `mealpy` library on two transportation problems, one of which has a known global optimum (Section 9); and a fully reproducible open-source implementation, with an ablation configuration (CA-static, the adaptive layer disabled) carried throughout to isolate its contribution.

It is important to state the scope of our claims clearly. We report results objectively and make no claim of universal superiority. Against the classical and widely used metaheuristics in the roster — GA, PSO, SA, GWO, WOA, and the six `mealpy` baselines of Section 9 — CA is consistently competitive and frequently significantly better, across synthetic suites, engineering design, and the transportation problems that motivated it. Against the two competition-grade optimizers,

the ordering is reversed: L-SHADE and CMA-ES outperform CA on every problem class tested, and this is reported without qualification wherever it occurs. The claim this paper defends is therefore deliberately narrower than a performance claim: CA is a well-founded, transparent, and competitive architecture whose every mechanism is specified mathematically, mapped to its operator family, and measured in isolation — and the ablation analysis of Section 7 identifies precisely which of those mechanisms carry its performance, a finding that in turn explains the standing of the specialized methods.

2 The Chess Algorithm

2.1 Conceptual mapping

CA maintains a population of N candidate solutions $\mathbf{X}_i \in \Omega \subset \mathbb{R}^D$, treated as pieces playing "toward" the global optimum. Table 1 summarizes how the vocabulary of the game maps onto algorithmic mechanisms.

Table 1: Mapping between chess concepts and CA mechanisms.

Chess concept	Algorithmic role
King	Incumbent best solution (never lost)
Queen, Rook, Bishop, Knight, Pawn	Heterogeneous movement operators
Development (opening principle)	Time-decaying step-scale $a(t)$
Sacrifice	Probabilistic acceptance of worse moves (minor pieces only)
Pinning	Progressive freezing of coordinates to the King's values
Castling	Safeguarded coordinate-block exchange King $\leftrightarrow$ Rook
En passant	Opportunistic, self-adaptive local capture around the King
Pawn promotion	Rank-based role reassignment each iteration
Threefold repetition	Re-deployment of agents that collapse onto the King
Opening / middlegame / closed position / endgame	Position-dependent tactical phase, read each iteration (Section 2.11)
Nimzowitsch prophylaxis	Restrained, line-reopening response to premature convergence
Fork, interference, discovered attack, royal council	Additional movement and probe operators, active in specific phases
Windmill	Repeated extension of a successful King displacement (endgame)
Zugzwang / blockade	Stagnation-triggered waiting move, King's march, and speculative sacrifice
Checkmate	Termination criterion

One point deserves emphasis: CA borrows the strategic grammar of chess as an inductive bias, not the game itself. An iteration costs $\mathcal{O}(ND + N \log N)$ arithmetic operations (the movement updates plus the sort that drives role assignment) and $N + 3$ function evaluations — N piece moves plus three en-passant probes — with one extra castling probe every c iterations. That is

the same order as GA, PSO, or GWO. The exact evaluation accounting used in the experiments is spelled out in Section 3.

2.2 Initialization

The board is set up by scattering the N pieces uniformly over the feasible box:

$$\mathbf{X}_i^{(0)} = \mathbf{l} + (\mathbf{u} - \mathbf{l}) \circ \mathbf{r}_i, \qquad \mathbf{r}_i \sim \mathcal{U}(0,1)^D, \quad i = 1, \dots, N, \tag{2}$$

where $\mathbf{l}$ and $\mathbf{u}$ are the bound vectors and $\circ$ is the elementwise (Hadamard) product. All positions are evaluated once and the population is sorted by fitness before the first move is played.

2.3 Role assignment

Let π be the permutation of $\{1, \dots, N\}$ that sorts the population,

$$f(\mathbf{X}_{\pi(1)}) \le f(\mathbf{X}_{\pi(2)}) \le \dots \le f(\mathbf{X}_{\pi(N)}). \tag{3}$$

The best agent $\mathbf{K} = \mathbf{X}_{\pi(1)}$ is the King. The following ranks are partitioned, in order, into Queens $\mathcal{Q}$, Rooks $\mathcal{R}$, Bishops $\mathcal{B}$, and Knights $\mathcal{N}$, with

$$|\mathcal{Q}| = \max(1, \lfloor \rho_q N \rceil), \quad |\mathcal{R}| = \max(1, \lfloor \rho_r N \rceil), \quad |\mathcal{B}| = \max(1, \lfloor \rho_b N \rceil), \quad |\mathcal{N}| = \max(1, \lfloor \rho_n N \rceil),$$

where $\lfloor \cdot \rceil$ denotes rounding and $(\rho_q, \rho_r, \rho_b, \rho_n) = (0.10, 0.15, 0.15, 0.20)$. Whatever remains becomes the Pawns $\mathcal{P}$. Because the assignment is redone after every iteration, a pawn that reaches a good region becomes a strong piece the next time roles are assigned. Promotion, in other words, falls out of re-ranking for free; no extra machinery is needed.

2.4 Development schedule

Opening theory tells a player to develop pieces quickly; endgames reward patience and precision. CA encodes this with a linearly decaying step-scale coefficient,

$$a(t) = 2\left(1 - \frac{t}{T}\right), \tag{4}$$

with t the current iteration and T the total budget — the convention popularized by Mirjalili et al. (2014). Every piece move scales with $a(t)$, so the swarm plays big developing moves early and fine maneuvers late.

2.5 Piece movement operators

Write $\mathbf{L} = \mathbf{u} - \mathbf{l}$ for the box width and let $\mathbf{r}, \mathbf{r}' \sim \mathcal{U}(0,1)^D$ be independent random vectors. The five mobile roles then move as follows.

Queen (omnidirectional sweep). The queen circles the King at a radius tied to her current distance from him:

$$\mathbf{X}_i' = \mathbf{K} + a(t)\,(2\mathbf{r} - \mathbf{1}) \circ \max(|\mathbf{K} - \mathbf{X}_i|,\ \sigma), \tag{5}$$

where σ is the King's adaptive capture radius from Section 2.6.3. The max keeps the sweep from collapsing entirely, so queens never stall.

Rook (single-axis move). One random axis j is chosen and only that coordinate moves:

$$X_{i,j}' = K_j + a(t)\,(2r - 1)\ \max(|K_j - X_{i,j}|,\ 0.01\,L_j\,a(t)). \tag{6}$$

Bishop (diagonal move). Two distinct axes j and k receive steps of equal magnitude and random relative sign, which is as close to a diagonal as a box-constrained space allows:

$$\Delta = a(t)\,(2r - 1)\ \max\Big(\tfrac{1}{2}(|K_j - X_{i,j}| + |K_k - X_{i,k}|),\ 0.01\,L\,a(t)\Big), \quad X_{i,j}' = K_j + \Delta, \quad X_{i,k}' = K_k \pm \Delta. \tag{7}$$

Knight (L-shaped jump). The knight is the one piece that jumps over whatever stands in its way. Relative to a randomly chosen better-ranked peer $\mathbf{P}$, it drifts toward the peer and then adds an asymmetric 2:1 jump on two random axes:

$$\mathbf{X}_i' = \mathbf{X}_i + \mathbf{r} \circ (\mathbf{P} - \mathbf{X}_i), \qquad X_{i,j}' \mathrel{+}= 2\,a(t)\,\beta\,(2r - 1), \quad X_{i,k}' \mathrel{+}= 1\,a(t)\,\beta\,(2r' - 1), \tag{8}$$

with $\beta = \overline{|\mathbf{P} - \mathbf{X}_i|}$ the mean coordinate gap. With small probability $0.1\,a(t)/2$ the knight abandons this move and leaps: two coordinates are resampled uniformly in Ω. This leap is the algorithm's main long-range restart device. The implementation also ships a heavy-tailed variant in which those two coordinates instead receive a Lévy-flight step $0.05\,L\,s$ generated by Mantegna's algorithm (Mantegna, 1994), the mechanism Cuckoo Search made popular (Yang & Deb, 2009):

$$s = \frac{u}{|v|^{1/\beta_\ell}}, \qquad u \sim \mathcal{N}(0, \sigma_u^2), \quad v \sim \mathcal{N}(0,1), \qquad \sigma_u = \left[\frac{\Gamma(1+\beta_\ell)\,\sin(\pi\beta_\ell/2)}{\Gamma(\frac{1+\beta_\ell}{2})\,\beta_\ell\,2^{(\beta_\ell - 1)/2}}\right]^{1/\beta_\ell}, \tag{9}$$

with tail index $\beta_\ell = 1.5$. Every result in this paper uses the plain uniform leap; the Lévy option is documented for future study rather than used here.

Pawn (steady advance). Pawns take small steps toward the King and toward a random better-ranked piece $\mathbf{B}$:

$$\mathbf{X}_i' = \mathbf{X}_i + 0.3\,\mathbf{r} \circ (\mathbf{K} - \mathbf{X}_i) + 0.3\,\mathbf{r}' \circ (\mathbf{B} - \mathbf{X}_i). \tag{10}$$

2.6 Strategic mechanisms

2.6.1 Pinning

With probability $p_{\text{pin}}(t) = 0.5\,t/T$, a random subset of at most 30% of an agent's coordinates is pinned to the King's values and excluded from perturbation. Early on, pinning is rare and the pieces develop freely. Late in the run it concentrates the search on a shrinking set of free dimensions around the incumbent, consistent with the exploitation schedule expected during an endgame phase.

2.6.2 Sacrifice

A worsening move $\Delta_i = f(\mathbf{X}_i') - f(\mathbf{X}_i) > 0$ made by a minor piece — a Knight or a Pawn — is still accepted with probability

$$p_{\text{acc}} = \exp\!\left(-\frac{\Delta_i}{\theta(t)}\right), \qquad \theta(t) = \theta_0 (f_{\max} - f_{\min})\, e^{-8t/T}, \tag{11}$$

with $\theta_0 = 0.1$. Readers will recognize the Metropolis rule of Simulated Annealing (Kirkpatrick et al., 1983), but two chess-flavored restrictions change its character. The temperature is scaled by the current fitness spread of the population, which makes the sacrifice budget self-calibrating across problems of wildly different magnitudes. And only minor pieces may be sacrificed: the King and the major pieces are never lost, so the elite of the population cannot be eroded by the acceptance rule.

2.6.3 En passant (adaptive local capture)

Once per iteration the King attempts an en-passant capture. Three trial points are drawn in a sparse Gaussian neighborhood,

$$\mathbf{Y}_m = \mathbf{K} + \sigma \circ \mathbf{z}_m \circ \mathbf{m}_m, \qquad \mathbf{z}_m \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad m = 1, 2, 3, \tag{12}$$

where the random binary mask $\mathbf{m}_m$ activates each coordinate with probability $\max(0.2,\ 2/D)$. Any improving trial replaces the King. The radius σ follows a success rule of the kind long used in Evolution Strategies (Beyer & Schwefel, 2002): multiply by 1.3 after a successful capture, by 0.92 after a failure, and after 50 consecutive failures reset to $\sigma = 0.05\,\mathbf{L}\max(a(t), 0.05)$ — a stalemate-avoidance restart. In our experience this mechanism is what keeps CA improving deep into the run, long after a fixed-radius local search would have gone quiet.

2.6.4 Castling

Every $c = 10$ iterations the King castles with the best Rook: a contiguous block of up to $D/4$ coordinates is copied from the Rook into a candidate King, and the exchange is kept only if it improves the King. It is a safeguarded structural jump, useful when good partial solutions have been discovered on different "files" of the board and need splicing together.

2.6.5 Threefold repetition (diversity preservation)

If an agent coincides with the King to numerical precision — something pinning can eventually cause — it is re-deployed uniformly in Ω, much as the threefold-repetition rule stops a game from going in circles. We did not add this rule for elegance; early prototypes taught us it is essential. Without it, pinning breeds exact clones of the King, every gap-proportional step length collapses to zero, and the search dies quietly in place.

2.6.6 Checkmate

The algorithm stops after T iterations. A stagnation-based early stop would be easy to add, but we keep it disabled in all experiments so that every algorithm consumes the same core budget of $N \times T$ population evaluations.

2.7 Parameter summary

Table 2 collects the base parameters and the single configuration used everywhere in this paper; several of them (pinning probability, the knight-leap rate, and others) are further modulated by the phase-conditioned schedule of Table 3 once the adaptive control layer of Section 2.11 is active — CA-static uses the values below unmodulated throughout. No per-problem tuning was done, for CA, for CA-static, or for any competitor.

Table 2: CA parameters and the fixed configuration used in all experiments.

Parameter	Symbol	Value
Role fractions (Queen / Rook / Bishop / Knight)	$\rho_q, \rho_r, \rho_b, \rho_n$	0.10/0.15/0.15/0.20
Development schedule	$a(t)$	$2(1 - t/T)$
Pinning probability / cap	$p_{\text{pin}}(t)$	$0.5\,t/T$, at most 30% of coordinates
Sacrifice constant	θ_0	0.1
Castling period	c	10 iterations
En-passant probes per iteration	—	3
En-passant adaptation (success / failure / reset)	—	$\times 1.3$ / $\times 0.92$ / after 50 failures
Knight exploratory-leap distribution	—	uniform (Lévy, $\beta_\ell = 1.5$, optional)

2.8 Pseudocode

```
Algorithm: Chess Algorithm (CA)
Input: objective f, bounds [l, u], dimension D,
       population N, iterations T
1  Initialize X and its board-mirror l+u-X; evaluate both; keep the
```

```
     fitter N (Eq. init); sort; King ← X(1); archive ← {}
2    ← 0.1 L; fails ← 0; stall ← 0; d_ema, acc_ema, eps_ema ← defaults
3  for t = 0 … T-1:
4      a ← 2(1 - t/T)
5      Read position: d_ema ← smoothed mean distance to King;
         phase ← Opening / Middlegame / Closed / Endgame (Eq. phase);
         if population idle (low acc_ema, eps_ema): halve a (Zugzwang)
6      Look up phase-conditioned rates: p_pin, leap mult., p_council,
         p_discovered-attack, p_interference, pawn gain (Table 3)
7      Assign roles by rank: Queens 10%, Rooks 15%, Bishops 15%,
         Knights 20%, Pawns rest
8      Move each piece by its operator; Knights fork or leap, Bishops
         occasionally interfere, Pawns advance at the phase's gain
9      Pin  30% of coordinates of each agent to King with prob. p_pin
10     Clip to bounds; evaluate F
11     Accept improvements; accept worse MINOR pieces with prob. e^(-Δ/ ),
          set from the robust population spread (Eq. sacrifice)
12     En passant: 3 trials around King - masked Gaussian, plus a
         council or discovered-attack probe with prob. p_council /
         p_discovered-attack, plus a single-coordinate check probe;
         keep any improvement; adapt  ; on success in Endgame, extend
         the winning displacement (windmill) while it keeps improving
13     Every 10 iters: Castle - block-swap King best Rook, keep if better
14     Threefold repetition: re-deploy agents identical to the King
15     Update King; re-rank population (pawn promotion); archive the
         King if it improved and is far from the current archive
16     if stall  25 (no significant improvement, Eq. fifty-move-rule):
         Pawn break (re-deploy all Pawns); King's march (6 long-radius
         probes, greedy-refine the best, keep if it improves the King);
         reheat the sacrifice budget ×10 for 10 iterations; replace the
         worst Queen with a random archived strongpoint; reset stall
17 return King
```

2.9 Flowchart

Figure 1 summarizes the core loop common to every iteration; the phase-dependent tactic selection and the blockade response of Section 2.11 sit inside its "apply role moves" and "update King" boxes respectively, and are given in full in the pseudocode above.

2.10 Properties and design rationale

Elitism and convergence. The King's fitness never increases: it is replaced only by strictly better points, whether through the population update, en passant, or castling. That elitism buys the usual asymptotic guarantee (Proposition 2.1).

Proposition 2.1 (Elitist convergence in probability)**.** *Let f be continuous on the compact box Ω with global minimum f^*. Under CA's update rules, the best-so-far value $f(\mathbf{K}^{(t)})$ converges in probability to f^* as $T \to \infty$.*

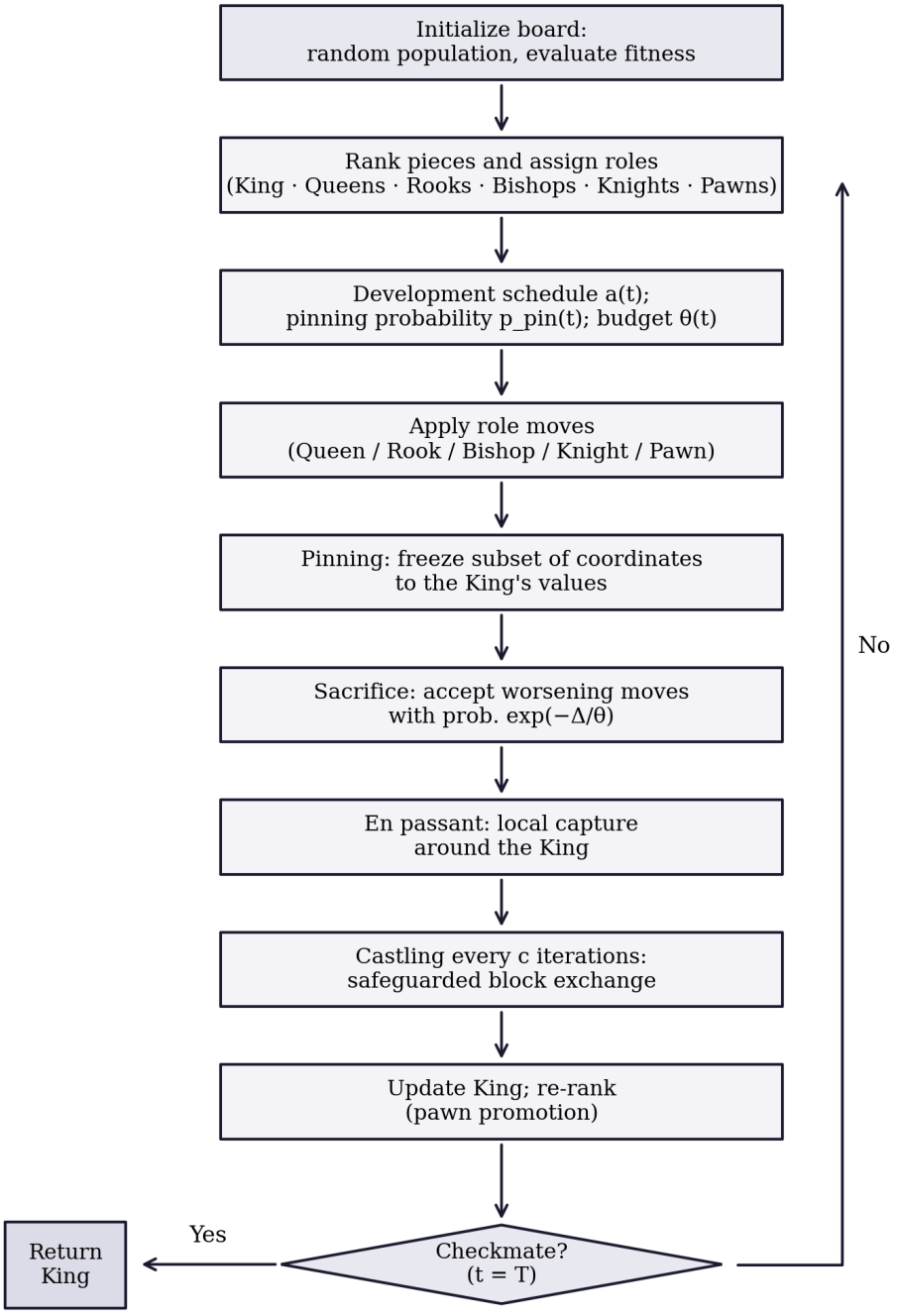

Figure 1: Flowchart of the Chess Algorithm.

Proof sketch. *The King sequence is elitist by construction. At every iteration the Knight's exploratory leap and the threefold-repetition re-deployment both place uniform probability mass over Ω, so any open subset — in particular any neighborhood of a global minimizer, whose objective values come within ε of f^* by continuity — is visited with probability approaching one as iterations accumulate. Elitism plus this covering argument yields convergence in probability, exactly as in the standard analysis of elitist stochastic search.*

An asymptotic statement of this kind provides no information about behavior within a finite budget of 15,000 evaluations; that question is empirical and is addressed in Section 3.

Exploration–exploitation balance. Exploration rests mainly on the Knights (Equation 8) and on the Queens early in the schedule (Equation 5); exploitation on the Pawns (Equation 10), on pinning, and on the en-passant success rule (Equation 12). The balance shifts smoothly through $a(t)$, $p_{\text{pin}}(t)$, and $\theta(t)$, and adapts to the state of the search through the gap-proportional step lengths and the spread-scaled sacrifice budget (Equation 11).

Parameter economy. Every result in this paper is produced under the single configuration in Table 2. The sensitivity of performance to the role fractions has not yet been quantified; this is identified explicitly as a direction for future work rather than omitted from discussion.

2.11 Adaptive tactical control

The mechanisms above give CA its movement vocabulary; what remains is to say how CA decides, moment to moment, which of them to lean on. A grandmaster does not run the same combination of tactics in a wide-open middlegame as in a locked pawn structure or a simplified endgame — which tactic is right depends on the position, not on a move counter. CA answers this with a lightweight state machine that reads four cheap statistics of the population each iteration and switches the active tactical repertoire accordingly, described below. For controlled comparison, every experiment in this paper is also run under **CA-static**: the identical operator set with this state machine disabled, so that role-appropriate tactics fire at fixed, non-adaptive rates instead. CA-static is not a separate algorithm; it is an ablation configuration, used throughout Section 4 and Section 6 to isolate exactly what the adaptive layer contributes.

2.11.1 Reading the position

Four exponentially-smoothed statistics ($\lambda = 0.8$) are recomputed every iteration. The population's normalized divergence from the King,

$$d(t) = \frac{1}{\text{Lspan}\sqrt{D}} \cdot \frac{1}{N-1} \sum_{i=2}^{N} \|\mathbf{X}_i^{(t)} - \mathbf{K}^{(t)}\|_2, \qquad \tilde{d}(t) = 0.8\,\tilde{d}(t-1) + 0.2\,d(t), \tag{13}$$

answers "is the position open or closed?", where $\text{Lspan} = \max_j(u_j - l_j)$. Two further exponential moving averages track the initiative,

$$\alpha(t) = 0.8\,\alpha(t-1) + 0.2 \cdot \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}[\text{move}_i \text{ accepted}], \tag{14}$$

and the technical success rate of the King's own local search,

$$\varepsilon(t) = \begin{cases} 0.8\,\varepsilon(t-1) + 0.2 & \text{en-passant capture succeeded at } t, \\ 0.8\,\varepsilon(t-1) & \text{otherwise.} \end{cases} \tag{15}$$

Finally, a stagnation counter borrows chess's *fifty-move rule*: rather than reset on any improvement whatsoever, it resets only on a materially significant one, exactly as fifty moves without a pawn push or a capture — not merely without a game-theoretically perfect move — are needed to claim a draw:

$$s(t) = \begin{cases} 0, & f(\mathbf{K}^{(t)}) < f_{\text{ref}} - \max(10^{-4}|f_{\text{ref}}|,\ 10^{-10}), \\ s(t-1) + 1, & \text{otherwise,} \end{cases} \tag{16}$$

with f_{ref} updated to $f(\mathbf{K}^{(t)})$ whenever $s(t) = 0$. Early prototypes used a naive counter that reset on any improvement whatsoever; because the en-passant rule of Section 2.6.3 produces microscopic refinements routinely, that counter never accumulated sufficiently to trigger the blockade response of Section 2.11.4, even in runs that remained visibly confined to a secondary basin for hundreds of iterations. Equation 16 addresses this limitation.

2.11.2 Game phases

The smoothed divergence and the fraction of the budget elapsed, $\phi(t) = t/T$, jointly select one of four phases:

$$\text{phase}(t) = \begin{cases} \textsc{Opening}, & \tilde{d}(t) > 0.22, \\ \textsc{Middlegame}, & 0.045 < \tilde{d}(t) \le 0.22, \\ \textsc{Closed}, & \tilde{d}(t) \le 0.045 \text{ and } \phi(t) \le 0.5, \\ \textsc{Endgame}, & \tilde{d}(t) \le 0.045 \text{ and } \phi(t) > 0.5. \end{cases} \tag{17}$$

The two thresholds were checked, though not exhaustively swept, against four combinations ($\delta_{\text{open}} \in \{0.15, 0.22, 0.30\}$, $\delta_{\text{end}} \in \{0.03, 0.045, 0.06, 0.09\}$) on two CEC-2017 functions (Rastrigin and Hybrid Function 4, $D = 30$); $\delta_{\text{open}} = 0.22$ and $\delta_{\text{end}} = 0.045$ were the best or near-best combination on both and are used throughout this paper. A full sensitivity study across the complete parameter space, noted as a limitation in Section 10, remains future work.

The Closed phase answers a failure mode found during tuning: a population can collapse to a small radius around the King *early* in the search — a premature convergence, not a genuine endgame — and treating every low-divergence state as an endgame made this worse, since endgame tactics (heavy pinning, aggressive pawn advance) accelerate collapse rather than reversing it. Reading an early collapse as a **prematurely closed position** and answering with Nimzowitschian *prophylaxis* — restraint and reopening lines rather than immediate commitment — removes the pathology. Each phase activates a different parameter vector, summarized in Table 3.

Table 3: Phase-conditioned tactical schedule. Pinning probability p_{pin}, knight-leap probability multiplier, council probability p_{c}, discovered-attack probability p_{DA}, interference probability p_{intf}, and pawn step gain, by phase. *Endgame pinning is capped at 0.5.

Phase	p_{pin}	Leap mult.	p_{c}	p_{DA}	p_{intf}	Pawn gain
Opening	0	2.0	0	0	0.30	0.30
Middlegame	$0.5\,\phi(t)$	1.0	0.5	0	0.15	0.30
Closed	0	2.0	0.5	0.15	0.15	0.30
Endgame	$0.1{+}0.75\,\phi(t)^*$	0.5	0.7	0.15	0	0.45

One overlay applies regardless of phase. If the population has lost the initiative and the King's local search has gone quiet at the same time,

$$\text{Zugzwang}(t) \iff \phi(t) > 0.2 \ \wedge \ \alpha(t) < 0.04 \ \wedge \ \varepsilon(t) < 0.05, \tag{18}$$

the development coefficient is halved for that one iteration, $a(t) \leftarrow a(t)/2$ — a *triangulation* move: change nothing structurally, spend a tempo, and let the next reading of the position be more informative before committing to a new phase.

2.11.3 New tactical operators

Novotny interference (Bishops). Named for the classical motif in which a piece is sacrificed on a square that blocks two enemy lines of defense at once, a Bishop is occasionally interposed on the segment between two distant, higher-ranked pieces rather than moving diagonally around the King:

$$\mathbf{X}_i' = \beta\,\mathbf{X}_a + (1-\beta)\,\mathbf{X}_b + 0.02\,a(t)\,\mathbf{L}\circ\mathbf{z}, \qquad \beta \sim \mathcal{U}(0.3, 0.7), \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \tag{19}$$

with a, b drawn without replacement from the fitter half of the population (probability p_{intf}, Table 3). Being a convex recombination of two arbitrary population members rather than a step along a coordinate axis, it is rotation-invariant, and it is the mechanism most directly responsible for CA's gains on the CEC-2017 hybrid and composition functions (Section 4).

The knight's fork. With probability 0.5, a Knight that does not take its exploratory leap instead plays

$$\mathbf{X}_i' = \mathbf{X}_i + \mathbf{r}\circ(\mathbf{K} - \mathbf{X}_i) + 0.8\,(\mathbf{X}_a - \mathbf{X}_b), \qquad a, b \sim \mathcal{U}\{1, \dots, i-1\}, \quad \mathbf{r} \sim \mathcal{U}(0,1)^D, \tag{20}$$

drifting toward the King while inheriting the differential structure of two better-ranked peers at once — attacking two targets simultaneously, in the spirit of the tactic that gives the move its name.

Discovered attack. An early version of this operator applied the geometry below to every Rook as a population move; tuning showed it tactically unsound at that scale, since the candidate is accepted by the ordinary improvement rule almost every time it is tried, collapsing the Rook sub-population onto the King–elite line within a handful of iterations. Restricted instead to a

single, strictly elitist probe competing for one of the King's own en-passant trials — active only in the Closed and Endgame phases —

$$\mathbf{Y}_{\mathrm{DA}} = \mathbf{X}_e + u\,(\mathbf{K} - \mathbf{X}_e), \qquad u \sim \mathcal{U}(1.2, 2.2), \quad e \sim \mathcal{U}\{1, 2, 3\}, \tag{21}$$

it recovers the intended effect — probing the far side of the incumbent along an elite-King line — without the pathology. This finding generalizes: a tactic that is sound for a single elite piece can destabilize an entire role class, analogous to how a discovered attack that wins material for one player becomes disadvantageous if applied by every piece on the board simultaneously.

Royal council. Competing with the discovered attack for the same probe slot (mutually exclusive, selected by relative probability),

$$\mathbf{Y}_{\mathrm{council}} = \mathbf{K} + 1.5\,u\,(\mathbf{K} - \mathbf{c}), \qquad \mathbf{c} = \tfrac{1}{2}(\mathbf{X}_{\mathrm{Q}_1} + \mathbf{X}_{\mathrm{R}_1}), \quad u \sim \mathcal{U}(0, 1), \tag{22}$$

reflects the King away from the midpoint of the best Queen and best Rook. Where isotropic Gaussian probes are blind to the shape of the elite, this reflection is population-shaped: when the elite lies strung along a narrow curved valley — the typical geometry along an active inequality constraint in the engineering problems of Section 6 — it points down the valley rather than across it. This single mechanism was responsible for closing most of the welded-beam gap observed in early prototypes.

Windmill. In the Endgame phase only, a successful en-passant or council capture with displacement $\delta = \mathbf{K}^{(t)} - \mathbf{K}^{(t)}_{\mathrm{pre}}$ is extended,

$$\mathbf{K} \leftarrow \mathbf{K} + \delta \quad \text{while } f(\mathbf{K} + \delta) < f(\mathbf{K}), \quad \text{up to 3 repetitions}, \tag{23}$$

echoing the chess windmill's repeating sequence of discovered checks, each one harvesting material.

Overprotection archive. Aron Nimzowitsch's overprotection principle counsels defending a valuable point with more force than strictly necessary, so that pieces freed from other duties have a strong square to retreat to. CA archives up to five mutually distant past King positions,

$$\text{admit } \mathbf{K}^{(t)} \iff f(\mathbf{K}^{(t)}) < f(\mathbf{K}^{(t-1)}) \ \wedge \ \min_{\mathbf{a} \in \mathcal{A}} \frac{\|\mathbf{K}^{(t)} - \mathbf{a}\|}{\mathrm{Lspan}\sqrt{D}} > 0.15, \tag{24}$$

evicting the worst by fitness on overflow. These strongpoints are the fallback used by the blockade response below.

2.11.4 Blockade: pawn break, King's march, and Tal's speculative sacrifice

When the fifty-move counter reaches $s(t) \geq 25$ (Equation 16), three mechanisms fire together. All Pawns are re-deployed uniformly at random — a *pawn break*, opening lines in a closed position. The King probes six long-radius candidates at three scales,

$$\mathbf{Y}_m = \mathbf{K} + \rho_m\,\mathbf{L} \circ \mathbf{z}_m, \qquad \rho_m \in \{0.1, 0.1, 0.2, 0.2, 0.4, 0.4\}, \quad \mathbf{z}_m \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \tag{25}$$

then *consolidates* the best candidate with an eight-step greedy local refinement of shrinking radius before comparing it against the incumbent King. The order is deliberate: refine-then-compare lets the march accept a basin whose raw landing point is worse than the incumbent but whose refined optimum is better, while the final elitist comparison still guarantees the King's fitness sequence never worsens. Simultaneously, the sacrifice budget of Equation 11 is reheated tenfold for the next ten iterations — modeled on Mikhail Tal's willingness to sacrifice material for an unquantifiable positional initiative, rather than continuing to apply conservative moves that have ceased to yield progress — and, if the archive of Equation 24 is non-empty, the worst-ranked Queen is replaced by a random archived strongpoint, giving the population a proven foothold rather than a blind restart.

2.11.5 Opening principle: opposition-based initialization

The initial population is evaluated together with its reflection through the board's center, $\mathbf{X}^{(0)}_{\text{mirror}} = \mathbf{l} + \mathbf{u} - \mathbf{X}^{(0)}$, and the fitter half of the combined $2N$ candidates is kept. Isolating this mechanism during tuning revealed a *mixed* effect — better on two of four micro-benchmark problems tested, worse on the other two — rather than a consistent gain. It is kept by default because it never produced a large regression and costs only one extra population evaluation, paid once; its dedicated ablation is reported in Section 7.1.

2.12 Relation to established operator families

The criticism of metaphor-driven algorithm design articulated by Sörensen (2015) deserves a direct answer rather than a defensive one: a vocabulary borrowed from a game can obscure the fact that the underlying operators are established ones, can impede comparison with prior work, and can inflate claims of novelty. The response this paper owes the reader is a precise disclosure of what each mechanism is mathematically and where it sits in the existing taxonomy. Table 4 provides that disclosure for every CA mechanism.

Table 4: Every CA mechanism, its established operator family, and the canonical reference for that family.

CA mechanism	Operator family	Canonical reference
Role-based moves (Queen, Rook, Bishop, Knight, Pawn)	Elite-guided stochastic perturbation, heterogeneous step geometry	Kennedy & Eberhart (1995); Mirjalili et al. (2014)
Development schedule $a(t)$	Time-decreasing step-size scheduling	Mirjalili et al. (2014)
Sacrifice (Equation 11)	Metropolis acceptance with population-scaled temperature	Kirkpatrick et al. (1983)
En passant σ rule (Equation 12)	Success-rule step-size adaptation, (1+λ)-ES	Beyer & Schwefel (2002); Hansen & Ostermeier (2001)
Knight's fork (Equation 20)	Differential mutation (current-to-best with difference vector)	Storn & Price (1997)
Novotny interference (Equation 19)	Arithmetic (convex) recombination	Goldberg (1989)

CA mechanism	Operator family	Canonical reference
Royal council (Equation 22)	Simplex-type reflection away from an elite centroid	Nelder & Mead (1965)
Discovered attack (Equation 21)	Line extrapolation through the incumbent	Nelder & Mead (1965)
Windmill (Equation 23)	Success-directed displacement repetition	Beyer & Schwefel (2002)
Pinning	Block-coordinate fixation (axis-aligned subspace exploitation)	—
Castling	Segment (block) crossover with elitist acceptance	Holland (1975)
Threefold repetition	Diversity maintenance by re-initialization	Goldberg (1989)
Blockade response (Section 2.11.4)	Partial restart with temperature reheating	Kirkpatrick et al. (1983)
Phase state machine; zugzwang (Section 2.11)	Adaptive parameter control	Eiben et al. (1999)
Opposition-based initialization	Opposition-based learning	Tizhoosh (2005)

Stated in these terms, CA is a composition of operators from the evolutionary-computation, swarm, and trajectory traditions, coordinated by an adaptive control layer and organized around a heterogeneous-role architecture. The chess vocabulary contributes no operator that the table cannot name; what it contributes is a compact, internally consistent specification language for a particular *composition* — which operators are present, at what rates, under which population states — and that composition, not any single operator, is the algorithmic claim of this paper. Two consequences follow. First, every mechanism named in Table 1 is defined mathematically in this section and measured in isolation in Section 7.1, so the metaphor never substitutes for specification or for evidence. Second, the mapping generates a falsifiable expectation: if CA's performance rests principally on a small subset of these operator families, then specialized algorithms built purely on those families should match or exceed CA where their assumptions hold. The roster of Section 4 through Section 9 includes L-SHADE (Tanabe & Fukunaga, 2014) and CMA-ES (Hansen & Ostermeier, 2001) — the competition-refined embodiments of the differential-mutation and step-size-adaptation families respectively — precisely to test that expectation, and Section 7.1 returns to it quantitatively.

3 Benchmark Experiments

3.1 Experimental protocol

CA is compared against its own non-adaptive ablation, CA-static (Section 2.11), and four established metaheuristics: a real-coded GA with tournament selection, BLX-α crossover, Gaussian mutation, and elitism (Goldberg, 1989; Holland, 1975); PSO with linearly decreasing inertia (Kennedy & Eberhart, 1995); SA (Kirkpatrick et al., 1983), granted a population-equivalent evaluation budget so the comparison stays fair; and GWO (Mirjalili et al., 2014).

The suite consists of six standard functions covering unimodal, ill-conditioned, and highly multimodal terrain (Table 5):

Table 5: Benchmark functions ($D = 30$).

Function	Type	Search domain	f^*
F1 Sphere	Unimodal	$[-100, 100]^{30}$	0
F2 Rosenbrock	Unimodal, ill-conditioned valley	$[-30, 30]^{30}$	0
F3 Rastrigin	Multimodal	$[-5.12, 5.12]^{30}$	0
F4 Griewank	Multimodal	$[-600, 600]^{30}$	0
F5 Ackley	Multimodal	$[-32, 32]^{30}$	0
F6 Schwefel 2.22	Unimodal, non-separable norm	$[-10, 10]^{30}$	0

Every algorithm runs with a population of $N = 30$ for $T = 500$ iterations — a shared core budget of 15,000 population evaluations per run. In the interest of full transparency: CA additionally spends three en-passant probes per iteration, one castling probe every ten iterations, and occasional windmill and blockade probes (Section 2); its measured consumption is 12.4% above the core budget (exact accounting in Section 7.3). The gaps reported below span orders of magnitude, so this overhead cannot be what drives them; note also that GWO beats CA *despite* it. Each algorithm–function pair is repeated over 30 independent runs, and all algorithms share the same seed at a given run index, so everyone faces the same initial conditions. No parameters were tuned per problem for any method.

Statistics follow Derrac et al. (2011): one-way ANOVA to establish that the algorithm factor matters at all, then pairwise two-sided Wilcoxon rank-sum tests (CA against each competitor) at $\alpha = 0.05$, which make no normality assumption about the final-error distributions.

3.2 Descriptive results

Table 6 reports the mean, standard deviation, best, and worst final errors for every algorithm–function pair.

Table 6: Mean, standard deviation, best, and worst final objective values over 30 runs (best mean per function in bold).

Function	Algorithm	Mean	Std	Best	Worst
Sphere	CA	6.159e-05	1.152e-04	2.149e-07	4.760e-04
Sphere	CA-static	4.630e-06	4.253e-06	2.111e-07	1.572e-05
Sphere	GA	9.475e-02	5.399e-02	2.890e-02	2.946e-01
Sphere	PSO	8.455e-01	7.216e-01	1.194e-01	3.131e+00
Sphere	SA	1.013e+04	2.218e+03	6.628e+03	1.612e+04
Sphere	GWO	**4.678e-31**	8.194e-31	6.431e-33	3.925e-30
Rosenbrock	CA	7.756e+01	4.383e+01	2.431e+01	1.912e+02
Rosenbrock	CA-static	5.513e+01	3.969e+01	7.695e+00	1.414e+02
Rosenbrock	GA	1.056e+02	4.633e+01	2.895e+01	2.863e+02
Rosenbrock	PSO	6.445e+03	2.238e+04	8.896e+01	9.021e+04
Rosenbrock	SA	2.185e+07	5.693e+06	7.615e+06	3.486e+07
Rosenbrock	GWO	**2.683e+01**	6.174e-01	2.592e+01	2.796e+01
Rastrigin	CA	1.953e+01	8.020e+00	7.960e+00	4.477e+01

Function	Algorithm	Mean	Std	Best	Worst
Rastrigin	CA-static	1.924e+01	4.897e+00	9.951e+00	3.383e+01
Rastrigin	GA	1.954e+01	6.823e+00	1.008e+01	4.187e+01
Rastrigin	PSO	5.971e+01	1.484e+01	3.306e+01	9.545e+01
Rastrigin	SA	2.105e+02	2.951e+01	1.598e+02	2.553e+02
Rastrigin	GWO	**1.023e+01**	1.078e+01	5.684e-14	4.680e+01
Griewank	CA	8.177e-03	7.777e-03	2.282e-05	2.473e-02
Griewank	CA-static	1.123e-02	1.143e-02	1.617e-05	3.705e-02
Griewank	GA	2.816e-01	7.971e-02	1.217e-01	4.364e-01
Griewank	PSO	6.164e-01	2.367e-01	2.902e-02	1.025e+00
Griewank	SA	9.190e+01	2.324e+01	4.826e+01	1.563e+02
Griewank	GWO	**4.303e-03**	9.094e-03	0.000e+00	3.591e-02
Ackley	CA	6.049e-02	3.189e-01	1.561e-04	1.778e+00
Ackley	CA-static	3.902e-02	2.073e-01	1.508e-04	1.155e+00
Ackley	GA	7.757e-02	1.486e-02	4.767e-02	1.099e-01
Ackley	PSO	1.045e+00	5.491e-01	8.837e-02	2.341e+00
Ackley	SA	4.483e+00	3.618e-01	3.807e+00	5.247e+00
Ackley	GWO	**3.325e-14**	4.369e-15	2.176e-14	4.308e-14
Schwefel 2.22	CA	1.554e-03	6.928e-04	3.858e-04	3.011e-03
Schwefel 2.22	CA-static	8.415e-04	3.824e-04	3.074e-04	1.562e-03
Schwefel 2.22	GA	7.438e-02	1.815e-02	4.451e-02	1.138e-01
Schwefel 2.22	PSO	7.941e-01	2.486e+00	3.962e-02	1.010e+01
Schwefel 2.22	SA	1.121e+07	2.877e+07	1.033e+03	1.180e+08
Schwefel 2.22	GWO	**9.928e-19**	9.462e-19	1.034e-19	4.919e-18

3.3 Convergence behavior

Figure 2 shows the median best-so-far curves, from which three patterns are apparent. On F1, F4, and F6, CA continues to improve throughout the run — the en-passant success rule continues to identify improvements deep into the later iterations, whereas GA and PSO stagnate hundreds of iterations earlier. On the ill-conditioned Rosenbrock valley (F2), convergence for every population method slows substantially, with CA and GA tracking each other closely throughout. CA and CA-static track each other almost exactly on every function, visually confirming the near-total absence of a significant difference between them on this suite. GWO exhibits its well-documented strength on this classical suite, converging fastest and to the lowest values throughout. The distributions of final values underlying these curves are shown in Figure 3.

3.4 Statistical tests

One-way ANOVA confirms that the algorithm factor is highly significant on every function (Table 7):

Table 7: One-way ANOVA over the six algorithms, per function.

Function	F-statistic	p-value
Sphere	605.079	5.569e-108

Function	F-statistic	p-value
Rosenbrock	427.018	1.130e-95
Rastrigin	777.692	5.369e-117
Griewank	451.683	1.229e-97
Ackley	951.036	2.680e-124
Schwefel 2.22	4.403	8.433e-04

The pairwise Wilcoxon tests give the sharper picture (Table 8):

Table 8: Wilcoxon rank-sum tests, CA versus each competitor ($\alpha = 0.05$).

Function	Comparison	p-value	Result (= 0.05)
Sphere	CA vs CA-static	1.480e-03	CA worse
Sphere	CA vs GA	2.872e-11	CA better
Sphere	CA vs PSO	2.872e-11	CA better
Sphere	CA vs SA	2.872e-11	CA better
Sphere	CA vs GWO	2.872e-11	CA worse
Rosenbrock	CA vs CA-static	6.043e-02	not significant
Rosenbrock	CA vs GA	1.086e-03	CA better
Rosenbrock	CA vs PSO	2.552e-09	CA better
Rosenbrock	CA vs SA	2.872e-11	CA better
Rosenbrock	CA vs GWO	2.954e-08	CA worse
Rastrigin	CA vs CA-static	6.681e-01	not significant
Rastrigin	CA vs GA	5.154e-01	not significant
Rastrigin	CA vs PSO	6.373e-11	CA better
Rastrigin	CA vs SA	2.872e-11	CA better
Rastrigin	CA vs GWO	6.160e-05	CA worse
Griewank	CA vs CA-static	7.007e-01	not significant
Griewank	CA vs GA	2.872e-11	CA better
Griewank	CA vs PSO	2.872e-11	CA better
Griewank	CA vs SA	2.872e-11	CA better
Griewank	CA vs GWO	1.479e-05	CA worse
Ackley	CA vs CA-static	1.099e-02	CA worse
Ackley	CA vs GA	5.317e-10	CA better
Ackley	CA vs PSO	4.403e-10	CA better
Ackley	CA vs SA	2.872e-11	CA better
Ackley	CA vs GWO	2.872e-11	CA worse
Schwefel 2.22	CA vs CA-static	5.101e-05	CA worse
Schwefel 2.22	CA vs GA	2.872e-11	CA better
Schwefel 2.22	CA vs PSO	2.872e-11	CA better
Schwefel 2.22	CA vs SA	2.872e-11	CA better
Schwefel 2.22	CA vs GWO	2.872e-11	CA worse

3.5 Discussion

Considered together, the tables support a differentiated conclusion rather than a single summary statement. Against PSO and SA, CA is significantly better on all six functions. Against GA it wins five of six — on Sphere, Rosenbrock, Griewank, Ackley, and Schwefel 2.22 — while

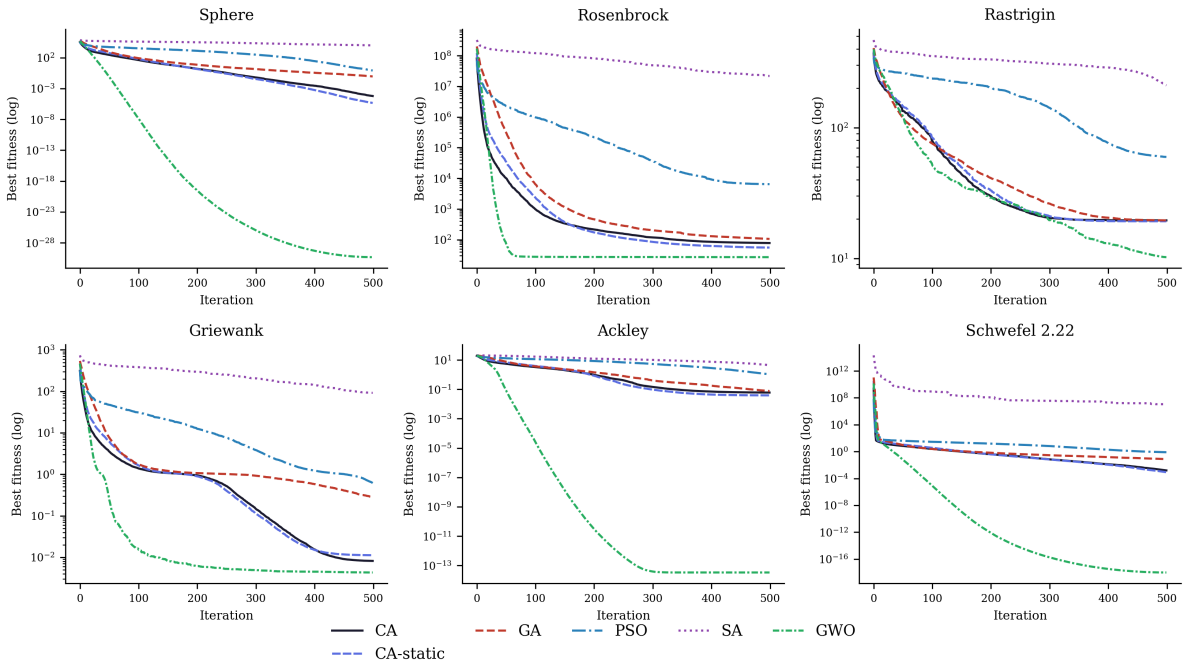

Figure 2: Median convergence curves (30 runs, semi-log scale).

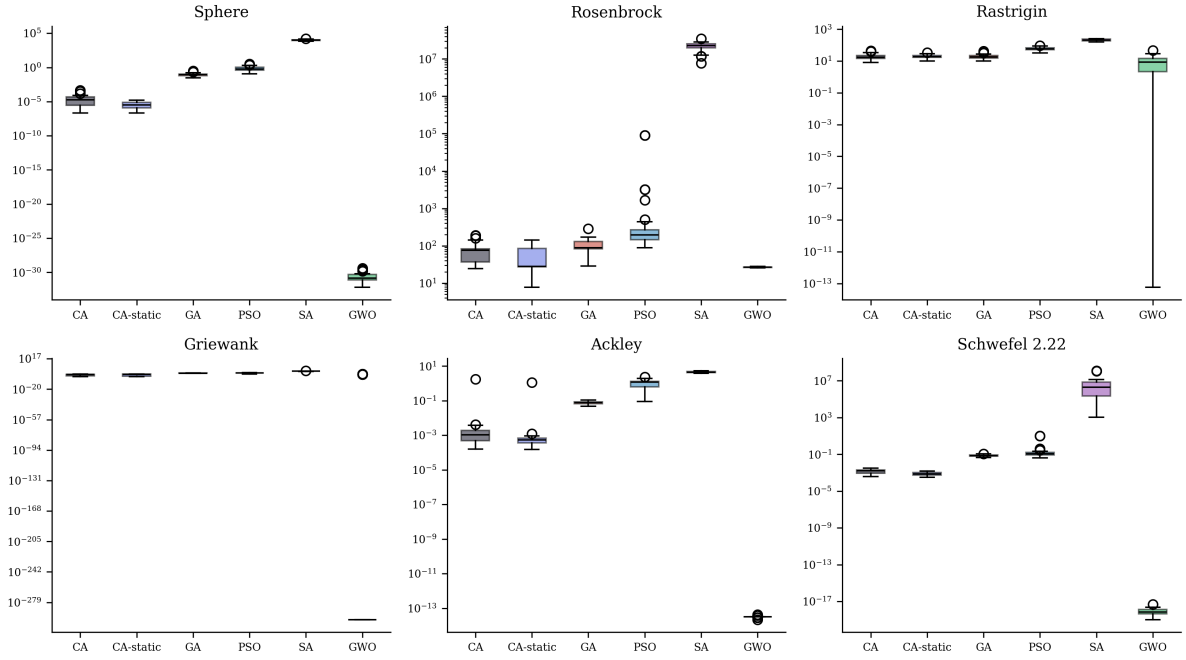

Figure 3: Distribution of final objective values over 30 runs.

Rastrigin ends in a statistical tie. Against GWO, the outcome is reversed: GWO is significantly better on all six classical benchmarks, consistent with the aggressive exploitation it is known for on exactly this suite (Mirjalili et al., 2014). We objectively acknowledge this limitation: on standard unconstrained test functions with central global optima, CA does not surpass GWO's aggressive exploitation capabilities.

The comparison against CA-static warrants particular attention, as it contradicts the initial expectations underlying this design. On this classical, low-structure suite, disabling the adaptive layer of Section 2.11 never hurts and sometimes *helps*: CA-static is significantly better than CA on Sphere, Ackley, and Schwefel 2.22, and statistically tied on the other three, with not a single function where the adaptive layer wins outright. This is interpreted as an informative negative result: the phase-reading machinery is built to recognize open, closed, and endgame *structure*, and a smooth unimodal or mildly multimodal function in a fixed box offers it little of that structure to exploit — the state machine spends its overhead reading a position that does not change much. The CEC-2017 and engineering-design landscapes of Section 4 and Section 6 are exactly where that overhead should start paying for itself, and whether it does is the empirical question those sections answer.

The practical implication is that these rankings are problem-dependent — precisely the situation Wolpert & Macready (1997) predicts — so the relevant question is not whether CA outperforms competitors on the Sphere function specifically, but whether it holds up on a harder, standardized synthetic suite and on structured engineering problems. That is where we go next: Section 4 extends the comparison to the full CEC-2017 suite, Section 6 turns to constrained engineering design, and Section 8 returns to the transportation motivation that opened the paper — and the ranking does change.

4 The Full CEC-2017 Benchmark Suite

4.1 Motivation and roster

The six-function suite of Section 3 is standard but limited in scope, and current conventions in this literature — including those established by our own baseline, GWO (Mirjalili et al., 2014) — require that a substantive claim of competitiveness be validated on a larger, more challenging, and independently curated suite. CEC-2017 meets this requirement: it comprises 29 usable functions, shifted, rotated, and in many cases constructed by hybridizing or composing several base functions, such that an algorithm cannot exploit coordinate-axis alignment or a single basin shape.[1]

We evaluate CA (Section 2.11) and its non-adaptive ablation CA-static against six competitors: GWO, PSO, and GA (our own implementations from Section 3); WOA (Mirjalili & Lewis, 2016) (third-party, `mealpy` defaults, as in Section 9); and two competition-grade adaptive optimizers included in response to the legitimate criticism that new metaheuristics are too often benchmarked only against classical or metaphor-derived baselines (Sörensen, 2015) — L-SHADE (Tanabe & Fukunaga, 2014), the success-history adaptive differential-evolution lineage that has defined the state of the art on CEC suites since 2014, via the `niapy` implementation (Vrbančič

[1]The official suite defines thirty functions, of which F2 (Shifted and Rotated Sum of Different Power) was withdrawn from the competition owing to numerical instability (Awad et al., 2016). The `opfunu` library renumbers the remaining twenty-nine consecutively, and this paper's labels follow that numbering; label Fn therefore corresponds to the official function F(n+1) for $n \geq 2$. The distinction matters for the cross-library audit of Section 4.2, whose second implementation retains the official numbering.

et al., 2018), and CMA-ES (Hansen & Ostermeier, 2001), the canonical covariance-matrix-adaptation evolution strategy, via Hansen's reference implementation `pycma` (Hansen et al., 2019). All algorithms run at $D = 30$ with population 30, 500 iterations, and 30 independent runs per algorithm–function pair, run r of every algorithm sharing seed $\text{SEED}_0 + r$ on the unit hypercube (real bounds folded into the decoder, as throughout this paper). Budget parity for the two additions is handled explicitly: CMA-ES runs $\lambda = 30$ for 500 generations, exactly the 15,000-evaluation core budget, and L-SHADE — whose linear population-size reduction is intrinsic to the algorithm, shrinking the population from 30 toward 4 as the budget is consumed — is capped at 15,500 evaluations, within the measured consumption of CA itself (Section 7.3), so neither addition holds a budget advantage over any incumbent.

4.2 Data integrity: auditing the reference implementation

CEC-2017 was accessed through `opfunu` v1.0.1, a third-party Python port of the official suite, and two independent checks were applied before any result was trusted. First, every candidate function was evaluated at the library's own reported global optimizer $\mathbf{x}^*$, requiring $|f(\mathbf{x}^*) - f^*| \leq 10^{-6} \max(1, |f^*|)$; this is a necessary but not sufficient check, since it verifies the function only at one point and says nothing about the rest of the domain. Second, we required the landscape to be discriminative: F5 (Shifted-and-Rotated Schaffer F7) passed the first check exactly ($f(\mathbf{x}^*) = f^* = 500.0$) but random samples spanning the entire search box returned values between 500.15 and 500.55 — a surface so nearly flat that no optimizer, ours or any competitor's, could extract a meaningful gradient from it.

A function can pass both of those checks and still be broken elsewhere in the domain. Five further functions were caught only empirically, during optimization itself, because at least one algorithm in the roster — and in the worst cases nearly every algorithm, including the weakest — returned a best-of-30-runs value definitionally impossible under a correct f^*. Four surfaced in the original six-algorithm runs; a fifth, F16, surfaced only when the stronger eight-algorithm roster was added, CMA-ES locating points 78 units below the claimed optimum — an illustration of why stronger baselines also strengthen data auditing. Table 9 collects the evidence for all six defective labels; the defective raw numbers are committed in the repository (`results/cec2017_opfunu_defect_evidence.csv`) rather than quietly dropped.

Table 9: Integrity audit of the `opfunu` CEC-2017 port: the six labels excluded, with the observed evidence of an implementation defect in each case.

Label	`opfunu` function	Claimed f^*	Defect observed
F5	Shifted and Rotated Schaffer F7	500.0	near-flat: random samples across the box span only $[500.12, 500.80]$ (std 0.13)
F9	Shifted and Rotated Schwefel	900.0	10-run CA probe found a best value of $-4{,}586.0$, a full 5,486 units below the claimed optimum

Label	`opfunu` function	Claimed f^*	Defect observed
F15	Hybrid Function 6	1500.0	best-of-run value −634.7 observed below claimed optimum
F16	Hybrid Function 7	1600.0	best-of-run value −78.1 observed below claimed optimum (surfaced only under the 8-algorithm roster)
F19	Hybrid Function 10	1900.0	best-of-run value −429.3 observed below claimed optimum
F21	Composition Function 2	2100.0	best-of-run value −9,592.4 observed below claimed optimum

Excluding defective implementations is only half of a defensible procedure; the other half is establishing whether the *functions themselves* can be evaluated correctly from an independent source. Each excluded label was therefore cross-validated against a second, independent Python implementation — `cec2017-py` (Tilley, 2020), adapted directly from the official C implementation and its reference data files, and retaining the official numbering (so paper label Fn is audited against official F(n+1); see the numbering footnote above). Each official counterpart was subjected to the full three-part audit: the one-point preflight, a landscape-discriminativeness check, and a ten-run below-optimum probe under the suite protocol. All six pass all three checks — including F5, whose official Schaffer F7 landscape is entirely discriminative (random-sample standard deviation of 40 against 0.1 in the `opfunu` port), and F9, whose official Schwefel landscape shows no below-optimum behavior whatever. Table 10 reports the audit in full.

Table 10: Restoration audit of the six excluded labels against the independent official-data implementation: one-point preflight error, random-sample spread, and the worst deviation below the official optimum across ten CA probe runs.

Label	Official func	f(shift+eps)	Official f*	Preflight err	Random-100 std	CA min (10 runs)	Worst below f*	Verdict
F5	F6	600.000	600.0	0.0000	40.06	629.883	+0.000	**Pass**
F9	F10	1000.000	1000.0	0.0004	1112	3542.222	+0.000	**Pass**
F15	F16	1600.002	1600.0	0.0018	6.172e+04	2057.193	+0.000	**Pass**
F16	F17	1700.000	1700.0	0.0001	5.061e+07	1919.152	+0.000	**Pass**
F19	F20	2000.023	2000.0	0.0231	477.8	2205.239	+0.000	**Pass**
F21	F22	2200.000	2200.0	0.0000	1074	2300.241	+0.000	**Pass**

The six labels were consequently restored using the independent implementation, under the identical protocol and seeds, and the analysis below covers all **29 validated functions** —

23 sourced from `opfunu` and 6 from the official-data port. Two conclusions from this audit generalize beyond this paper. Every defect traced to the `opfunu` port rather than to the suite itself, and the pattern of the defects (formulas paired with mismatched shift data) is consistent with a bookkeeping misalignment introduced by the post-withdrawal renumbering. And a data-integrity audit of this kind is not optional overhead: five of the six defects would silently corrupt any benchmark study that trusted the port, and one of them was only detectable because the roster contained algorithms strong enough to expose it.

4.3 Results

Table 11 reports the mean Friedman rank of each algorithm across all 29 validated functions, and Table 12 the win/tie/loss record of CA against each competitor under a two-sided Wilcoxon rank-sum test at $\alpha = 0.05$. The full per-function statistics are in Table 13; Figure 4 shows median convergence on six representative functions spanning unimodal, multimodal, hybrid, and composition landscapes.

Table 11: Mean Friedman rank across all 29 validated CEC-2017 functions (lower is better; $\chi^2 = 150.08$, $p = 3.9 \times 10^{-29}$).

Algorithm	Mean Friedman rank
L-SHADE	1.759
CMA-ES	2.276
CA	**3.621**
CA-static	3.897
GA	4.138
PSO	6.034
GWO	6.414
WOA	7.862

Table 12: CA win/tie/loss against each competitor, CEC-2017, 29 functions, Wilcoxon rank-sum at $\alpha = 0.05$.

Competitor	Win	Tie	Loss
CA-static	5	23	1
GWO	27	1	1
PSO	22	6	1
GA	10	9	10
WOA	29	0	0
L-SHADE	1	2	26
CMA-ES	6	3	20

Table 13: Mean, standard deviation, best, and worst final error $f - f^*$ over 30 runs, all 29 validated CEC-2017 functions (best mean per function in bold).

Func	Algorithm	Mean	Std	Best	Worst
F1	CA	**3011**	2587	197.2	1.223e+04
F1	CA-static	4186	4081	190.5	1.709e+04

Func	Algorithm	Mean	Std	Best	Worst
F1	GWO	4.3781e+09	3.6738e+09	1.2942e+07	1.4000e+10
F1	PSO	2.3283e+09	2.5019e+09	6.4555e+05	9.9419e+09
F1	GA	2.4003e+05	1.4751e+05	2.947e+04	6.9702e+05
F1	WOA	8.4880e+09	4.3850e+09	2.2548e+09	2.3894e+10
F1	L-SHADE	5476	5088	86.8	1.951e+04
F1	CMA-ES	6069	6549	83.66	2.07e+04
F2	CA	107.7	204.6	0.06075	780.8
F2	CA-static	3.805	6.16	0.02656	24.72
F2	GWO	1.564e+04	1.178e+04	2454	4.541e+04
F2	PSO	2742	5215	113.1	2.82e+04
F2	GA	10.85	5.971	2.576	26.3
F2	WOA	5.175e+04	1.826e+04	2.045e+04	8.45e+04
F2	L-SHADE	**0.001651**	0.002849	1.5418e-05	0.0153
F2	CMA-ES	0.009239	0.02618	2.2054e-09	0.1215
F3	CA	107.8	43.4	22.7	198.6
F3	CA-static	99.03	39.97	16.04	177.3
F3	GWO	411.1	447.5	42.44	1871
F3	PSO	310.5	445.3	35.29	2096
F3	GA	86.67	50.25	31.86	202.1
F3	WOA	1416	750.8	479.5	3244
F3	L-SHADE	38.69	18.2	20.11	90.38
F3	CMA-ES	**31.82**	0.03087	31.81	31.98
F4	CA	228.9	42.15	133.3	327.4
F4	CA-static	212.5	55.14	121.7	391
F4	GWO	6084	4170	688.8	1.571e+04
F4	PSO	4978	3476	1089	1.357e+04
F4	GA	148	35.3	87.11	219.2
F4	WOA	2.64e+04	8785	1.119e+04	5.035e+04
F4	L-SHADE	151.3	44.19	97.74	297.3
F4	CMA-ES	**29.55**	25.34	13.15	161.7
F5	CA	42.36	7.978	28.05	59.72
F5	CA-static	41.98	9.663	22.17	70.16
F5	GWO	26.35	11.08	7.722	57.04
F5	PSO	22.16	9.454	7.286	51.02
F5	GA	33.29	15.85	9.368	81.4
F5	WOA	105.4	14.87	80.66	145.2
F5	L-SHADE	10.27	3.944	3.704	19.07
F5	CMA-ES	**0.01966**	0.04062	1.1329e-04	0.1859
F6	CA	273.9	75.94	149.4	428.3
F6	CA-static	231.5	47.18	153.4	324.8
F6	GWO	8.224e+04	1.1049e+05	1.011e+04	5.2492e+05
F6	PSO	1.9461e+05	1.7735e+05	2.719e+04	8.9813e+05
F6	GA	312.4	52.84	229.3	412.1
F6	WOA	3.7174e+05	1.5982e+05	1.4091e+05	9.1497e+05
F6	L-SHADE	225.4	96.7	93.88	552.3
F6	CMA-ES	**76.04**	51.97	45.68	226.3
F7	CA	207.8	35.02	124	282
F7	CA-static	215.5	44.72	124	298
F7	GWO	377	140.5	111.9	604.1

Func	Algorithm	Mean	Std	Best	Worst
F7	PSO	255	77.89	125.4	413
F7	GA	251.2	66.57	118	358
F7	WOA	846.4	130.5	649.5	1142
F7	L-SHADE	99.19	26.99	60.2	177
F7	CMA-ES	**83.15**	29.41	54	206
F8	CA	4.821	2.025	2.004	12.04
F8	CA-static	5.035	2.34	0.9982	9.215
F8	GWO	6.806	4.302	0.8538	19.03
F8	PSO	4.802	2.868	0.1875	10.58
F8	GA	7.657	3.671	1.908	18.69
F8	WOA	25.43	10.2	11.36	43.89
F8	L-SHADE	1.358	0.991	4.4074e-09	3.903
F8	CMA-ES	**0.1453**	0.263	0	0.9086
F9	CA	3823	642.9	2542	5012
F9	CA-static	3747	635.8	2168	4602
F9	GWO	6687	1963	2352	8579
F9	PSO	3984	918.3	2205	6060
F9	GA	3618	708	1778	4915
F9	WOA	6692	1134	4190	8676
F9	L-SHADE	**2451**	350.6	1794	3396
F9	CMA-ES	4191	2609	696.2	7817
F10	CA	2.232e+04	9882	8453	5.258e+04
F10	CA-static	2.353e+04	1.528e+04	2787	6.693e+04
F10	GWO	2.9010e+05	7.887e+04	1.5099e+05	4.7707e+05
F10	PSO	3.4876e+05	8.7568e+05	1.247e+04	3.7428e+06
F10	GA	2.328e+04	1.348e+04	3055	6.502e+04
F10	WOA	3.4194e+05	7.9388e+05	6.97e+04	4.5950e+06
F10	L-SHADE	**2862**	2210	712.3	1.016e+04
F10	CMA-ES	1.682e+04	3.168e+04	108.8	1.0958e+05
F11	CA	3.2870e+05	2.6355e+05	3.439e+04	1.0288e+06
F11	CA-static	5.8859e+05	6.3353e+05	3.506e+04	2.6364e+06
F11	GWO	1.3921e+08	2.6493e+08	4.3572e+06	1.1956e+09
F11	PSO	2.2279e+08	4.7394e+08	8.507e+04	2.2180e+09
F11	GA	5.292e+04	3.32e+04	1.273e+04	1.7377e+05
F11	WOA	4.7012e+08	7.0769e+08	2.6239e+07	3.1975e+09
F11	L-SHADE	**1.989e+04**	2.208e+04	3948	1.0511e+05
F11	CMA-ES	9.959e+04	1.0406e+05	1.079e+04	5.8073e+05
F12	CA	2.525e+04	4.452e+04	3121	2.3720e+05
F12	CA-static	3.129e+04	4.555e+04	5391	2.5447e+05
F12	GWO	1.0450e+08	4.9388e+08	6.1448e+05	2.7589e+09
F12	PSO	1.7909e+08	5.0647e+08	3.127e+04	2.6839e+09
F12	GA	8973	6242	755.8	2.537e+04
F12	WOA	1.7695e+08	4.6631e+08	9.4148e+06	2.6417e+09
F12	L-SHADE	**6974**	5553	77.01	1.872e+04
F12	CMA-ES	5.567e+04	1.433e+04	2.296e+04	9.509e+04
F13	CA	2.2120e+05	1.8679e+05	1.834e+04	6.6846e+05
F13	CA-static	1.9711e+05	1.7575e+05	3.497e+04	7.7942e+05
F13	GWO	1.7826e+06	2.4331e+06	2.0212e+05	1.0381e+07
F13	PSO	4.5379e+05	4.2879e+05	9.481e+04	2.0864e+06

Func	Algorithm	Mean	Std	Best	Worst
F13	GA	2.8917e+06	2.4925e+06	4.78e+04	8.3697e+06
F13	WOA	5.2760e+06	4.6962e+06	2.4167e+05	1.5368e+07
F13	L-SHADE	**3.468e+04**	1.1790e+05	789.3	6.6576e+05
F13	CMA-ES	6.904e+04	6.861e+04	3443	2.4746e+05
F14	CA	2.36e+04	7888	6264	4.497e+04
F14	CA-static	3.082e+04	1.062e+04	1.202e+04	5.938e+04
F14	GWO	1.7884e+07	3.8538e+07	3.0152e+05	1.8339e+08
F14	PSO	1.0135e+07	4.1026e+07	1.687e+04	2.2348e+08
F14	GA	2.169e+04	1.141e+04	3974	5.219e+04
F14	WOA	5.6631e+07	8.3635e+07	2.8791e+05	2.8611e+08
F14	L-SHADE	**1.095e+04**	6551	1948	3.108e+04
F14	CMA-ES	4.704e+04	1.546e+04	1.968e+04	1.0199e+05
F15	CA	1012	285.4	457.2	1644
F15	CA-static	1042	310	379.7	1816
F15	GWO	1425	540.1	591.4	2431
F15	PSO	1135	331.1	488.2	1853
F15	GA	1114	302.2	452.1	1688
F15	WOA	2729	885.4	1528	5303
F15	L-SHADE	673.8	217.1	169.7	1126
F15	CMA-ES	**269.4**	179.1	32.58	638.6
F16	CA	445.4	184.3	90.54	832.5
F16	CA-static	419.8	194	84.11	830.5
F16	GWO	714.4	295.9	267	1387
F16	PSO	488.6	218	98.63	912
F16	GA	700.7	275.5	60.09	1311
F16	WOA	1564	467	931.3	2907
F16	L-SHADE	211.1	104.6	64.02	389.1
F16	CMA-ES	**176.1**	95.68	64.88	511.3
F17	CA	5.881e+04	4.279e+04	1.717e+04	2.3831e+05
F17	CA-static	4.293e+04	1.876e+04	1.042e+04	1.0299e+05
F17	GWO	1.3832e+05	5.843e+04	7.511e+04	3.9912e+05
F17	PSO	3.8353e+05	1.0266e+06	6.179e+04	5.6591e+06
F17	GA	7.1096e+05	7.7840e+05	1.339e+04	3.2841e+06
F17	WOA	1.8559e+05	3.4863e+05	3.658e+04	2.0331e+06
F17	L-SHADE	**1.302e+04**	1.287e+04	2498	6.918e+04
F17	CMA-ES	5.885e+04	3.515e+04	1.202e+04	1.4937e+05
F18	CA	1.463e+04	1.355e+04	251.5	5.897e+04
F18	CA-static	1.025e+04	1.217e+04	491.6	4.737e+04
F18	GWO	5.2931e+09	1.0298e+10	5.402e+04	5.0407e+10
F18	PSO	6.0458e+09	1.7771e+10	1.222e+04	8.6826e+10
F18	GA	2.014e+04	1.389e+04	4736	6.583e+04
F18	WOA	9.9471e+09	2.0025e+10	2.4809e+07	9.9822e+10
F18	L-SHADE	**424.2**	610.8	44.65	2890
F18	CMA-ES	8.523e+04	4.799e+04	1.388e+04	2.2372e+05
F19	CA	298.8	116.5	202.5	613.9
F19	CA-static	361.7	133.4	87.41	652.8
F19	GWO	673.1	308.7	215.5	1276
F19	PSO	402.3	160.6	205.2	822.5
F19	GA	662.6	246.6	71.76	1131

Func	Algorithm	Mean	Std	Best	Worst
F19	WOA	939.4	249.4	420.8	1534
F19	L-SHADE	264.4	116.7	70.43	494
F19	CMA-ES	**259.2**	186.8	60.75	762.5
F20	CA	363.1	175.6	100.1	694.9
F20	CA-static	291.4	184.7	100.1	597.7
F20	GWO	4150	5185	583.2	2.272e+04
F20	PSO	2035	1133	400.9	5567
F20	GA	287.8	107.8	110.5	396.2
F20	WOA	1.446e+04	1.082e+04	2128	3.639e+04
F20	L-SHADE	260.6	140.3	100	537.6
F20	CMA-ES	**221.1**	61.64	100	276.6
F21	CA	**346.4**	914.5	100.1	4087
F21	CA-static	728.9	1423	100.1	4688
F21	GWO	4445	3012	201.5	8750
F21	PSO	2070	1906	106.5	6213
F21	GA	3032	2011	103.3	5544
F21	WOA	6478	1179	4056	8178
F21	L-SHADE	774.2	1181	100	3487
F21	CMA-ES	2337	2063	100	7623
F22	CA	314.5	210.5	200.6	853.4
F22	CA-static	317.3	199.2	100.4	826.1
F22	GWO	9891	4098	1937	1.628e+04
F22	PSO	1.08e+04	5061	1584	2.012e+04
F22	GA	265.7	66.79	228.3	515.4
F22	WOA	1.72e+04	8057	6458	4.191e+04
F22	L-SHADE	**224.3**	93.96	200	664.4
F22	CMA-ES	248.9	70.27	200	392.3
F23	CA	224.5	94.6	200.3	676.9
F23	CA-static	272	150.3	200.3	720.6
F23	GWO	5984	2107	1626	9470
F23	PSO	5347	2684	559	9612
F23	GA	254.3	78.26	220.7	548.5
F23	WOA	1.296e+04	8319	4099	3.544e+04
F23	L-SHADE	**221.7**	65.19	200	433.2
F23	CMA-ES	229.2	52.99	200	336.3
F24	CA	451.7	22.39	428.2	531.4
F24	CA-static	443.2	18.05	424.2	501.8
F24	GWO	639.5	214.1	436.6	1614
F24	PSO	490	127.5	421.9	1067
F24	GA	435.7	18.28	422.5	502
F24	WOA	1038	314.7	665.6	2286
F24	L-SHADE	423.5	4.252	421	440.3
F24	CMA-ES	**420.5**	0.0608	420.4	420.7
F25	CA	866.9	45.29	788.2	1084
F25	CA-static	871.5	16.87	835.8	903.7
F25	GWO	913.7	30.28	847.7	980.6
F25	PSO	1052	145.2	865.3	1509
F25	GA	840.1	14.75	793.9	870.7
F25	WOA	2731	2035	912	1.034e+04

Func	Algorithm	Mean	Std	Best	Worst
F25	L-SHADE	840.3	17.29	825.7	873.6
F25	CMA-ES	**827.9**	2.203	825.4	832.5
F26	CA	547.1	31.32	500.7	600.7
F26	CA-static	558.8	49.5	473.4	681.5
F26	GWO	558.8	28.63	511.8	650.4
F26	PSO	588.6	58.75	501.8	786.7
F26	GA	523	22.3	470.9	563.9
F26	WOA	1231	240.7	830.8	1817
F26	L-SHADE	508.8	13.79	479	533.1
F26	CMA-ES	**508.4**	15.04	476.7	539.7
F27	CA	227.6	162.1	44.06	482.2
F27	CA-static	229.1	172.3	24.44	477.4
F27	GWO	609.3	182.8	475.7	1377
F27	PSO	810.5	322.4	275.2	1475
F27	GA	102.6	145.3	18.38	476.1
F27	WOA	1073	666.2	562.9	2837
F27	L-SHADE	128.1	110.5	29.12	448.7
F27	CMA-ES	**1.928**	4.377	0.001209	16.78
F28	CA	9.86e+04	1.3873e+05	1.587e+04	7.2781e+05
F28	CA-static	1.2912e+05	1.9055e+05	1.372e+04	8.3153e+05
F28	GWO	2.6902e+08	4.3159e+08	7.957e+04	1.8853e+09
F28	PSO	3.0455e+07	6.4775e+07	2.427e+04	2.8840e+08
F28	GA	1.4133e+05	2.3724e+05	3842	1.1699e+06
F28	WOA	8.6269e+08	1.0617e+09	1.7668e+07	4.1723e+09
F28	L-SHADE	7281	5508	2366	2.952e+04
F28	CMA-ES	**6086**	2085	1910	1.024e+04
F29	CA	8.8e+04	1.2333e+05	1.973e+04	6.9350e+05
F29	CA-static	2.2181e+05	2.5501e+05	2.74e+04	1.0666e+06
F29	GWO	5.0197e+08	1.1373e+09	1.4198e+05	4.2573e+09
F29	PSO	6.3055e+08	1.9914e+09	2.16e+04	9.3748e+09
F29	GA	6.5429e+05	1.3877e+06	8067	5.6345e+06
F29	WOA	3.1907e+09	5.4029e+09	5.7569e+06	2.3284e+10
F29	L-SHADE	**1.947e+04**	1.16e+04	6540	5.41e+04
F29	CMA-ES	1.2181e+05	9.095e+04	4.441e+04	4.9174e+05

4.4 Discussion

The two competition-grade methods lead decisively: L-SHADE takes the best mean Friedman rank (1.76) and CMA-ES the second (2.28), each beating CA on roughly ninety percent of functions (26/29 and 20/29 losses for CA respectively, with only 1 and 6 wins). This is reported as the central result of this suite, not a caveat to it — Section 7.1 explains it mechanistically. Among the remaining six algorithms, CA takes the best mean Friedman rank (3.62), narrowly ahead of CA-static (3.90) and GA (4.14), and well ahead of PSO (6.03), GWO (6.41), and WOA (7.86). Against GWO, PSO, and WOA, CA wins nearly all comparisons — 27/29, 22/29, and 29/29 respectively, with a single loss to each of GWO and PSO (both on F5, the restored Schaffer F7 function) and none to WOA. Against its own non-adaptive ablation, CA-static, the adaptive machinery of Section 2.11 wins 5 and loses 1 of 29, with 23 ties: a real but modest net gain, consistent with the expected effect of a well-tuned control layer applied to an

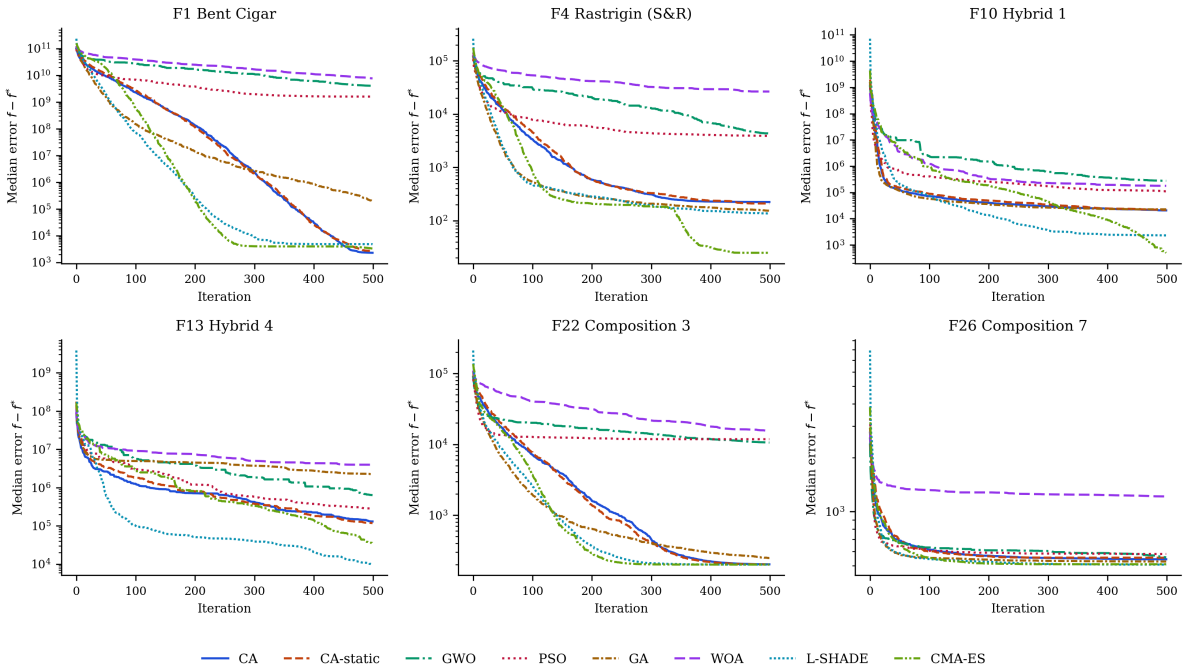

Figure 4: Median convergence on six representative CEC-2017 functions (30 runs, semi-log scale).

already-competitive base algorithm rather than a comprehensive replacement of it.

The comparison least favorable to CA among the classical/moderate roster is against GA, where the near-tied mean rank conceals a 10-win / 9-tie / 10-loss record function by function. The ten losses are not scattered at random: they are F4, F5, F11, F12, F20, F22, F24, F25, F26, and F27 — Rastrigin-family, hybrid, and composition landscapes whose basins are separated by distances comparable to the search domain itself. GA's BLX-α crossover routinely proposes offspring *outside* the interval spanned by two parents, an operator that can relocate a candidate across the entire domain in a single step. Every CA operator introduced in Section 2.11, by contrast, is anchored to the King, an elite peer, or a bounded local neighborhood; even the Knight's exploratory leap resamples only two of thirty coordinates. This outcome is interpreted as an illustration of the No-Free-Lunch theorem (Wolpert & Macready, 1997) rather than as a deficiency to be minimized: CA was tuned specifically against constrained engineering geometry and smooth hybrid/composition exploitation (Section 2.11), and that specialization has a measurable opportunity cost on landscapes whose defining difficulty is long-range, unstructured basin-hopping. We return to this trade-off, and to a concrete proposal for closing it, in Section 10.

5 The CEC-2022 Benchmark Suite

5.1 Motivation and protocol

CEC-2017 is the more extensive suite, but it is no longer the most recent one; a reviewer is entitled to ask whether the picture it paints survives on the current generation of benchmark design. CEC-2022 (Kumar et al., 2021) comprises twelve functions — Zakharov, Rosenbrock, expanded Schaffer F7, non-continuous Rastrigin, and Lévy under shift and rotation, three hybrid functions, and four composition functions — defined at two official dimensionalities,

$D = 10$ and $D = 20$, both of which are evaluated here. The roster is the full eight-algorithm set of Section 4: CA, CA-static, GWO, PSO, GA, WOA, L-SHADE, and CMA-ES. The protocol is likewise identical — population 30, 500 iterations, 30 independent runs, shared seed $\text{SEED}_0 + r$ at run index r, unit-hypercube search with bounds folded into the decoder. One departure from the official competition setup is disclosed rather than hidden: the CEC-2022 termination criteria specify budgets of 200,000 ($D = 10$) and 1,000,000 ($D = 20$) function evaluations, whereas this study retains its paper-wide core budget of 15,000 evaluations so that results remain comparable across every section; the findings below therefore characterize behavior under a modest, practice-oriented budget rather than at the official competition horizon.

5.2 Data integrity

The same two-stage audit as Section 4.2 was applied to `opfunu`'s CEC-2022 port. The preflight check passes for all twelve functions at both dimensionalities — unlike the CEC-2017 port. The post-hoc below-optimum audit, applied after the full runs with the same tolerance, nonetheless excluded six (function, dimension) cells in which at least one algorithm's best run fell decisively below the library's claimed optimum — F7 and F8 at $D = 20$, F10 at both dimensionalities, and F11 and F12 at $D = 10$, with observed dips ranging from -41 to $-4{,}588$ — again an implementation defect rather than a property of any algorithm, since in the worst cases nearly every algorithm in the roster, including the weakest, crossed the claimed optimum. Additionally, F3 (expanded Schaffer F7, the same function family whose near-flat CEC-2017 port motivated a flatness rule in Section 4.2) proved non-discriminative at both dimensionalities under this budget: the *worst-performing* algorithm's median final error is 0 at $D = 10$ and 0.066 at $D = 20$, so every method essentially solves it and it contributes no ranking information. Table 14 summarizes the exclusions; sixteen validated cells remain (eight per dimensionality), and the full pre-audit numbers are committed in the repository (`results/cec2022_stats_raw_unaudited.csv`).

Table 14: CEC-2022 data-integrity audit: excluded (function, dimension) cells and the reason for each exclusion.

Function	Reason excluded
F10-D10	below-optimum (opfunu defect)
F10-D20	below-optimum (opfunu defect)
F11-D10	below-optimum (opfunu defect)
F12-D10	below-optimum (opfunu defect)
F7-D20	below-optimum (opfunu defect)
F8-D20	below-optimum (opfunu defect)
F3-D10	near-flat (worst-mean PSO median final error $0 < 1.0$)
F3-D20	near-flat (worst-mean PSO median final error $0.0657 < 1.0$)

5.3 Results

Table 15 reports the mean Friedman rank of each algorithm across the sixteen validated cells, and Table 16 the win/tie/loss record of CA against each competitor under the two-sided Wilcoxon rank-sum test at $\alpha = 0.05$. Figure 5 shows median convergence on three representative functions per dimensionality; the full per-cell statistics are in Table 17.

Table 15: Mean Friedman rank across the 16 validated CEC-2022 (function, dimension) cells (lower is better; $\chi^2 = 81.69$, $p = 6.2 \times 10^{-15}$).

Algorithm	Mean Friedman rank
L-SHADE	1.875
CMA-ES	2.062
CA	**3.625**
CA-static	3.750
GA	4.750
PSO	5.625
GWO	6.438
WOA	7.875

Table 16: CA win/tie/loss against each competitor, CEC-2022, 16 validated cells, Wilcoxon rank-sum at $\alpha = 0.05$.

Competitor	Win	Tie	Loss
CA-static	1	13	2
GWO	12	4	0
PSO	12	2	2
GA	10	4	2
WOA	16	0	0
L-SHADE	1	4	11
CMA-ES	2	2	12

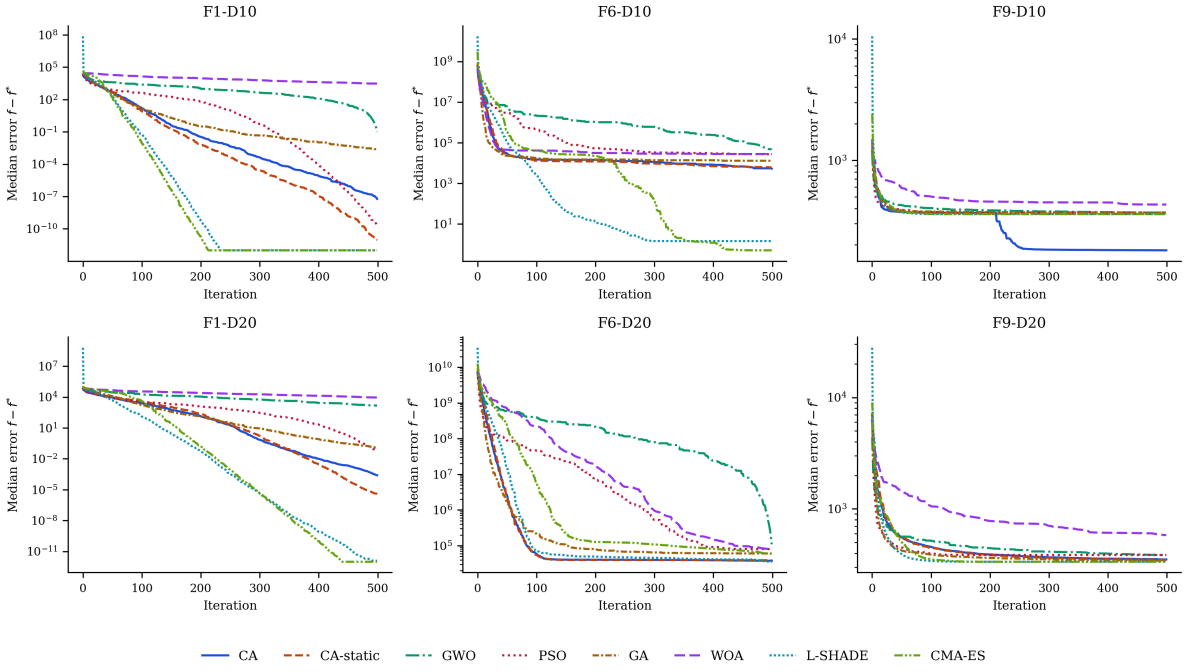

Figure 5: Median convergence on six representative CEC-2022 cells — F1 (Zakharov), F6 (Hybrid 1), and F9 (Composition 1) at each of the two official dimensionalities (30 runs, semi-log scale).

Table 17: Mean, standard deviation, best, and worst final error $f - f^*$ over 30 runs, all 16 validated CEC-2022 cells (best mean per cell in bold).

Func	Algorithm	Mean	Std	Best	Worst
F1-D10	CA	1.3919e-07	1.8316e-07	6.1567e-10	8.4307e-07
F1-D10	CA-static	1.1278e-07	4.5234e-07	5.6843e-14	2.4716e-06
F1-D10	GWO	267.4	560.4	0.03031	1579
F1-D10	PSO	6.6457e-10	1.0137e-09	7.0486e-12	4.2533e-09
F1-D10	GA	0.003916	0.005432	6.5056e-04	0.03035
F1-D10	WOA	4044	3642	568.6	1.627e+04
F1-D10	L-SHADE	2.4632e-14	2.8168e-14	0	5.6843e-14
F1-D10	CMA-ES	**0**	0	0	0
F2-D10	CA	14.36	25.33	1.7873e-04	73.52
F2-D10	CA-static	12.06	24.07	1.3643e-04	86.64
F2-D10	GWO	26	32.79	6.484	93.38
F2-D10	PSO	19.75	18.49	4.499	72.25
F2-D10	GA	10.57	18.76	0.05041	84.52
F2-D10	WOA	69.67	101	0.1597	361.5
F2-D10	L-SHADE	6.451	2.465	3.987	8.916
F2-D10	CMA-ES	**6.451**	2.465	3.987	8.916
F4-D10	CA	20.36	7.008	10	36.77
F4-D10	CA-static	22.6	7.36	10	38
F4-D10	GWO	44.42	12.01	18	66.25
F4-D10	PSO	31.27	9.64	16	48
F4-D10	GA	45.8	19.66	18	106
F4-D10	WOA	102.4	44.61	22	177.9
F4-D10	L-SHADE	18.47	6.212	8.001	34
F4-D10	CMA-ES	**17.8**	6.052	8	34
F5-D10	CA	0.0996	0.1728	2.2737e-13	0.5439
F5-D10	CA-static	0.1964	0.2853	7.7307e-12	0.9086
F5-D10	GWO	0.006198	0.02241	3.5462e-05	0.09016
F5-D10	PSO	0.01195	0.03043	1.5461e-11	0.08953
F5-D10	GA	0.7676	1.202	6.0990e-06	4.461
F5-D10	WOA	2.956	2.068	0.1818	8.348
F5-D10	L-SHADE	0.002984	0.01607	0	0.08953
F5-D10	CMA-ES	**0**	0	0	0
F6-D10	CA	6264	4765	121.5	1.661e+04
F6-D10	CA-static	8690	7381	389	2.986e+04
F6-D10	GWO	4.426e+04	1.849e+04	1.449e+04	7.483e+04
F6-D10	PSO	2.848e+04	1.561e+04	3396	7.232e+04
F6-D10	GA	1.716e+04	1.334e+04	877.7	4.869e+04
F6-D10	WOA	2.917e+04	1.893e+04	5062	6.768e+04
F6-D10	L-SHADE	**36.77**	186.8	0.1007	1043
F6-D10	CMA-ES	79.58	241.6	0.09689	955.6
F7-D10	CA	20.06	5.306	0.6316	29.52
F7-D10	CA-static	19.73	8.406	1.129	46.08
F7-D10	GWO	46.96	18.63	24.92	103.7
F7-D10	PSO	30.42	10.34	22.11	69.07
F7-D10	GA	25.73	11.99	1.984	49.26
F7-D10	WOA	374.9	315	32.58	1262
F7-D10	L-SHADE	**12.16**	9.212	1.175	22.13

Func	Algorithm	Mean	Std	Best	Worst
F7-D10	CMA-ES	27.69	13.29	3.382	63.71
F8-D10	CA	21.15	5.843	2.351	35.07
F8-D10	CA-static	19.16	6.819	0.9059	22.75
F8-D10	GWO	555.4	659.1	27.49	2945
F8-D10	PSO	407.2	894.8	6.389	2663
F8-D10	GA	56.54	100.1	2.85	439
F8-D10	WOA	1924	1229	266.5	4529
F8-D10	L-SHADE	**18.93**	5.97	0.7092	23.23
F8-D10	CMA-ES	20.19	7.353	2.144	26.41
F9-D10	CA	**183.8**	183.8	1.2777e-05	371.8
F9-D10	CA-static	208.3	182.2	2.4829e-09	373.6
F9-D10	GWO	375.3	27.2	360.6	518
F9-D10	PSO	328.9	141.2	1.6698e-06	465.1
F9-D10	GA	285.6	158.1	0.01728	445.6
F9-D10	WOA	416.5	140.4	7.589	729
F9-D10	L-SHADE	275.3	151.9	4.5475e-13	359.1
F9-D10	CMA-ES	359.1	1.7053e-13	359.1	359.1
F1-D20	CA	3.0095e-04	2.1270e-04	7.5344e-05	0.001173
F1-D20	CA-static	1.2142e-05	2.3606e-05	2.5607e-07	1.2641e-04
F1-D20	GWO	2182	2795	6.578	1.03e+04
F1-D20	PSO	766.9	3526	0.005895	1.968e+04
F1-D20	GA	0.1164	0.04908	0.02812	0.2446
F1-D20	WOA	1.277e+04	1.202e+04	771.7	4.872e+04
F1-D20	L-SHADE	3.2852e-11	1.0691e-10	5.6843e-14	5.8105e-10
F1-D20	CMA-ES	**1.0819e-12**	4.3343e-12	0	2.2908e-11
F2-D20	CA	62.75	18.66	1.478	94
F2-D20	CA-static	61.53	13.45	28.18	77.43
F2-D20	GWO	101.8	71.86	45.65	296.2
F2-D20	PSO	73.02	33.93	27.56	168.3
F2-D20	GA	60.24	12.07	45	80.72
F2-D20	WOA	226.4	125.1	81.83	762.8
F2-D20	L-SHADE	48.73	5.444	44.9	76.19
F2-D20	CMA-ES	**48.67**	1.257	44.9	49.08
F4-D20	CA	95.27	27.17	44	172
F4-D20	CA-static	110.2	28.16	52	176
F4-D20	GWO	184.4	68.6	48.04	268.9
F4-D20	PSO	129.1	39.35	66	198.6
F4-D20	GA	141.1	44.2	56	250
F4-D20	WOA	408.3	78.42	223.5	521
F4-D20	L-SHADE	59.64	17.84	30.01	106
F4-D20	CMA-ES	**43.87**	10.54	24	64
F5-D20	CA	1.588	0.8434	0.08956	4.272
F5-D20	CA-static	1.628	0.767	0.4544	3.995
F5-D20	GWO	1.311	1.174	0.001642	4.182
F5-D20	PSO	1.719	1.852	2.2923e-04	6.592
F5-D20	GA	3.179	2.803	0.1798	10.48
F5-D20	WOA	10.22	4.252	3.638	17.48
F5-D20	L-SHADE	0.2258	0.2831	5.6843e-13	0.9982
F5-D20	CMA-ES	**0.002984**	0.01607	0	0.08953

Func	Algorithm	Mean	Std	Best	Worst
F6-D20	CA	3.971e+04	1.667e+04	9909	9.508e+04
F6-D20	CA-static	4.312e+04	1.52e+04	2.237e+04	8.015e+04
F6-D20	GWO	1.1382e+06	3.6069e+06	4.267e+04	1.6021e+07
F6-D20	PSO	5.2067e+05	2.3587e+06	2.153e+04	1.3222e+07
F6-D20	GA	5.903e+04	2.376e+04	1.704e+04	1.0908e+05
F6-D20	WOA	7.7248e+05	3.2254e+06	3.668e+04	1.7967e+07
F6-D20	L-SHADE	**3.717e+04**	1.617e+04	1.266e+04	7.419e+04
F6-D20	CMA-ES	6.225e+04	1.933e+04	5231	1.0745e+05
F9-D20	CA	355.6	13.06	337.9	388.9
F9-D20	CA-static	338.8	63.68	0.002043	374
F9-D20	GWO	395.7	43	348.8	548.9
F9-D20	PSO	438	167.7	339.5	1253
F9-D20	GA	343.7	5.018	336.6	359.2
F9-D20	WOA	615.1	149.3	369.1	967.1
F9-D20	L-SHADE	335.6	7.7165e-05	335.6	335.6
F9-D20	CMA-ES	**335.6**	5.4072e-12	335.6	335.6
F11-D20	CA	4.998	9.974	0.002283	37.07
F11-D20	CA-static	152.5	430.6	3.7559e-05	1546
F11-D20	GWO	522.2	695.3	13.75	1825
F11-D20	PSO	356.1	558.8	12.32	1694
F11-D20	GA	8.723	11.57	0.02764	33.71
F11-D20	WOA	999.4	1076	27.14	3491
F11-D20	L-SHADE	49.69	246.6	5.9845e-10	1377
F11-D20	CMA-ES	**1.7451e-08**	3.3407e-08	1.2496e-09	1.6867e-07
F12-D20	CA	257.5	24.44	220	312.5
F12-D20	CA-static	259.3	29.23	223.8	323.1
F12-D20	GWO	269.8	27.88	231	337.1
F12-D20	PSO	295	44.46	233	385.3
F12-D20	GA	256.6	35.34	210.6	371.4
F12-D20	WOA	720.3	207.5	322.2	1143
F12-D20	L-SHADE	**233**	8.669	214.4	258.5
F12-D20	CMA-ES	239.2	16.99	215.9	289.2

5.4 Discussion

The ordering at the top of Table 15 is unambiguous and consistent with Section 4: the two competition-grade methods lead (L-SHADE 1.88, CMA-ES 2.06), followed by CA (3.63) and CA-static (3.75) as the best-ranked of the remaining six, then GA (4.75), PSO (5.63), GWO (6.44), and WOA (7.88). The pattern is stable across dimensionalities: restricting the Friedman analysis to the eight $D = 10$ cells gives L-SHADE first and CMA-ES second ($\chi^2 = 39.3$, $p = 1.7 \times 10^{-6}$), and restricting it to the eight $D = 20$ cells gives CMA-ES first and L-SHADE second ($\chi^2 = 44.1$, $p = 2.0 \times 10^{-7}$), with CA third at $D = 20$ and effectively tied with CA-static for third at $D = 10$.

The pairwise records make both directions of the comparison explicit. Against the classical roster, CA wins 12 of 16 cells against GWO and PSO, 10 against GA, and all 16 against WOA, with at most 2 losses to any of them. Against L-SHADE it loses 11 of 16 with a single win (F11, the second composition function, at $D = 20$); against CMA-ES it loses 12 with two wins (F7 at $D = 10$ and F6 at $D = 20$, both hybrid functions). These losses are reported as the central fact

of this section, not as a footnote to it: under this budget, the specialized adaptive machinery of L-SHADE and CMA-ES is decisively stronger on shifted-rotated synthetic landscapes than CA's chess-derived composition, at both dimensionalities. The NFL-based reading of such reversals, developed for the GA trade-off in Section 4, applies here in the same form and is not repeated; what is specific to this suite is that the GA trade-off itself does not reappear — CA beats GA 10-4-2 on CEC-2022, against a losing record on CEC-2017 — indicating that GA's long-range crossover advantage is specific to the 30-dimensional CEC-2017 hybrid and composition geometry rather than a general property. Section 7.1 offers a mechanism-level account of *why* the two specialized methods lead, which is more informative than the observation itself.

6 Constrained Engineering Design Benchmarks

6.1 Problems and roster

Alongside standardized synthetic suites, it is standard practice in this literature to validate a new metaheuristic on the small set of constrained engineering design problems that recur across numerous published comparisons, because a shared, literature-anchored problem enables direct comparison against independently produced results. We use seven such problems in their commonly published formulations: welded beam design, tension/compression spring design, pressure vessel design, speed reducer design, three-bar truss design, gear train design, and cantilever beam design. Constraints are handled by the same static-penalty approach used elsewhere in this paper (Section 9): $f_{\text{pen}}(\mathbf{x}) = f(\mathbf{x}) + 10^6 \sum_i \max(0, g_i(\mathbf{x}))^2$. Roster and protocol are identical to Section 4 (all eight algorithms; population 30, 500 iterations, 30 runs).

6.2 Results

Table 18 reports the mean Friedman rank of each algorithm across the seven problems, Table 19 the win/tie/loss record of CA against each competitor, and Table 20 the full per-problem statistics; Figure 6 shows median convergence on all seven.

Table 18: Mean Friedman rank across the 7 engineering design problems (lower is better; $\chi^2 = 33.87$, $p = 1.8 \times 10^{-5}$).

Algorithm	Mean Friedman rank
L-SHADE	1.571
CMA-ES	2.143
CA	**3.714**
CA-static	4.143
GWO	5.000
PSO	5.714
GA	6.143
WOA	7.571

Table 19: CA win/tie/loss against each competitor, engineering design problems, Wilcoxon rank-sum at $\alpha = 0.05$.

Competitor	Win	Tie	Loss
CA-static	1	6	0
GWO	3	4	0
PSO	4	2	1
GA	6	1	0
WOA	7	0	0
L-SHADE	0	0	7
CMA-ES	0	1	6

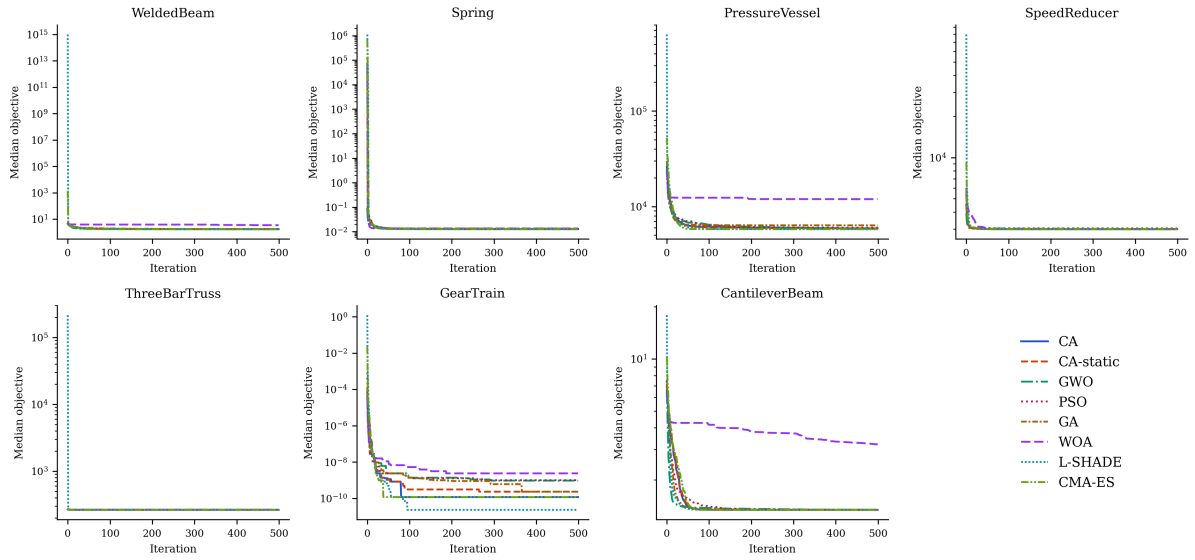

Figure 6: Median convergence on the seven engineering design problems (30 runs, semi-log scale).

Table 20: Mean, standard deviation, best, and worst objective over 30 runs, all 7 engineering design problems (best mean per problem in bold).

Problem	Algorithm	Mean	Std	Best	Worst
Welded Beam	CA	1.774	0.1085	1.725	2.157
Welded Beam	CA-static	1.778	0.1323	1.725	2.352
Welded Beam	GWO	1.728	0.001381	1.726	1.73
Welded Beam	PSO	1.725	0.001027	1.725	1.731
Welded Beam	GA	1.803	0.09243	1.726	2.068
Welded Beam	WOA	172.6	910.3	2.079	5075
Welded Beam	L-SHADE	**1.725**	6.9982e-16	1.725	1.725

Problem	Algorithm	Mean	Std	Best	Worst
Welded Beam	CMA-ES	1.725	2.0180e-15	1.725	1.725
Spring	CA	0.01293	3.1918e-04	0.01267	0.01373
Spring	CA-static	0.01305	8.5625e-04	0.01267	0.01645
Spring	GWO	0.01281	1.6745e-04	0.01267	0.01336
Spring	PSO	0.01307	9.3019e-04	0.01267	0.01777
Spring	GA	0.01378	0.001224	0.01269	0.01726
Spring	WOA	0.01362	8.4305e-04	0.01267	0.01574
Spring	L-SHADE	0.01267	2.3773e-05	0.01267	0.01277
Spring	CMA-ES	**0.01267**	8.9809e-06	0.01267	0.01271
Pressure Vessel	CA	5969	236.1	5787	6923
Pressure Vessel	CA-static	5959	230.6	5787	6792
Pressure Vessel	GWO	5887	191.1	5788	6627
Pressure Vessel	PSO	6076	297.9	5787	7301
Pressure Vessel	GA	6408	386.3	5834	7294
Pressure Vessel	WOA	1.621e+04	1.109e+04	6768	4.887e+04
Pressure Vessel	L-SHADE	5798	31.37	5787	5907
Pressure Vessel	CMA-ES	**5788**	0.1058	5787	5788
Speed Reducer	CA	**2994**	1.0100e-12	2994	2994
Speed Reducer	CA-static	2994	4.7938e-09	2994	2994
Speed Reducer	GWO	3000	3.543	2994	3008
Speed Reducer	PSO	3032	34.64	2994	3188
Speed Reducer	GA	2994	1.3233e-07	2994	2994
Speed Reducer	WOA	2994	1.129	2994	2999
Speed Reducer	L-SHADE	2994	6.2683e-13	2994	2994
Speed Reducer	CMA-ES	2994	4.5475e-13	2994	2994
Three-Bar Truss	CA	263.9	1.1794e-11	263.9	263.9
Three-Bar Truss	CA-static	263.9	1.1985e-11	263.9	263.9
Three-Bar Truss	GWO	263.9	0.004537	263.9	263.9

Problem	Algorithm	Mean	Std	Best	Worst
Three-Bar Truss	PSO	263.9	6.8562e-06	263.9	263.9
Three-Bar Truss	GA	263.9	0.001248	263.9	263.9
Three-Bar Truss	WOA	266.8	3.698	263.9	282.1
Three-Bar Truss	L-SHADE	**263.9**	5.6843e-14	263.9	263.9
Three-Bar Truss	CMA-ES	263.9	5.9618e-14	263.9	263.9
Gear Train	CA	7.3005e-10	1.0704e-09	2.7009e-12	4.5033e-09
Gear Train	CA-static	8.0825e-10	1.1062e-09	2.7009e-12	4.5033e-09
Gear Train	GWO	8.4991e-10	8.0675e-10	2.7009e-12	2.3576e-09
Gear Train	PSO	3.1678e-09	5.9167e-09	2.7009e-12	2.7265e-08
Gear Train	GA	7.9160e-10	9.9339e-10	2.7009e-12	4.5033e-09
Gear Train	WOA	2.2534e-08	5.7712e-08	2.7009e-12	3.0059e-07
Gear Train	L-SHADE	**5.8222e-11**	1.6460e-10	2.7009e-12	8.8876e-10
Gear Train	CMA-ES	9.3803e-10	1.3943e-09	2.7009e-12	6.5123e-09
Cantilever Beam	CA	1.34	4.0084e-05	1.34	1.34
Cantilever Beam	CA-static	1.34	1.9888e-05	1.34	1.34
Cantilever Beam	GWO	1.34	8.7514e-05	1.34	1.34
Cantilever Beam	PSO	1.34	4.3135e-05	1.34	1.34
Cantilever Beam	GA	1.342	0.001269	1.34	1.346
Cantilever Beam	WOA	3.372	1.118	1.621	6.243
Cantilever Beam	L-SHADE	**1.34**	0	1.34	1.34
Cantilever Beam	CMA-ES	1.34	0	1.34	1.34

6.3 Discussion

The two competition-grade methods lead this suite as they do the synthetic ones. L-SHADE takes the best mean Friedman rank (1.57), beating CA significantly on all seven problems; CMA-ES is second (2.14), beating CA on six with one statistical tie (gear train, where CA's mean is in fact fractionally lower). The terminal precision these two methods reach is instructive: L-SHADE attains the welded-beam literature optimum (1.72485) with a run-to-run standard deviation below 10^{-15}, and reproduces the best-known speed-reducer, three-bar-truss, and cantilever-beam values with essentially zero variance across all thirty runs — on this problem class, success-history parameter adaptation behaves as a reliable constrained refinement engine, and that level of terminal accuracy is one CA's fixed-rate probes do not reach. On the pressure vessel, both methods also descend below the literature reference value, which is admissible: the commonly cited 5,885.33 applies to the mixed-integer formulation in which two thicknesses are

multiples of 0.0625 in, whereas the continuous relaxation solved here (by every algorithm in the roster, identically) admits lower objectives.

Among the remaining six algorithms, CA is the best ranked (3.71, ahead of CA-static at 4.14, GWO at 5.00, PSO at 5.71, GA at 6.14, and WOA at 7.57). Against that classical roster CA loses exactly once across 35 pairwise comparisons — to PSO on the welded-beam problem, where PSO's near-zero run-to-run variance around the known optimum (1.7251 ± 0.0010) is difficult for any method that retains meaningful exploration to match on a low-dimensional, well-conditioned landscape. One additional observation merits explicit note: WOA under `mealpy`'s default hyperparameters is dramatically unstable on several of these constrained problems — its mean objective on welded beam is 172.6, two orders of magnitude above the optimum of 1.725, driven by a small number of catastrophically infeasible runs (Figure 6, top-left panel) — a reminder that default hyperparameters tuned for unconstrained synthetic benchmarks do not automatically transfer to constrained, penalty-shaped landscapes.

7 Component, Parameter, and Cost Analysis

The preceding sections treat CA as a unit. This section takes it apart: which of its mechanisms carry the performance (Section 7.1), how sensitive that performance is to the parameter configuration of Table 2 (Section 7.2), and what the algorithm costs in evaluations and wall-clock time, measured rather than estimated (Section 7.3). A composite algorithm with twelve named mechanisms invites the objection that most of them are ornamental; the analysis below answers that objection with measurements rather than assertion, and several of the answers are unflattering to individual components, which is what makes the remaining ones credible.

7.1 Granular ablation

Each of the twelve strategic mechanisms of Section 2 is disabled one at a time — all other mechanisms held at their published defaults — and each single-mechanism-off variant is compared against full CA over 30 runs on six problems spanning the paper's sections: CEC-2017 F1 (Bent Cigar), F4 (Rastrigin), F13 (Hybrid 4), and F22 (Composition 3) at $D = 30$ under the suite protocol and seeds of Section 4, the welded-beam problem under the protocol of Section 6, and the berth allocation problem P2 under the protocol of Section 9. Table 21 reports, for each mechanism, the ratio of the ablated variant's mean final error to full CA's, averaged over the six problems, together with the number of problems on which the difference is statistically significant; the per-problem breakdown is committed in the repository (`results/table_ablation_detail.md`).

Table 21: Granular ablation: each mechanism disabled in isolation, mean final-error ratio relative to full CA averaged over six problems (ratios above 1 indicate the mechanism helps), with Wilcoxon significance counts at $\alpha = 0.05$.

Mechanism disabled	Mean error ratio (ablated / full CA)	Significant on (of 6 problems)
En passant (success-rule local capture)	603.3	5
Knight's fork	46.842	5
Threefold repetition	1.213	3
Sacrifice	1.070	0

Mechanism disabled	Mean error ratio (ablated / full CA)	Significant on (of 6 problems)
Pinning	1.050	0
Windmill	1.043	0
Opposition-based initialization	1.030	1
Discovered attack	1.024	0
Blockade response	1.006	0
Novotny interference	0.999	0
Castling	0.999	0
Royal council	0.990	3

Two mechanisms carry most of the load, and they are not of equal kind. Disabling the en-passant capture — the success-rule local refinement of Section 2.6.3 — degrades the mean error by a factor of roughly 600 averaged over the six problems, dominated by a factor of 3.6×10^3 on Bent Cigar, and the degradation is significant on five of six problems including the welded beam (+1.9%, $p = 0.003$); CA's terminal accuracy on smooth or locally smooth landscapes rests essentially on this one mechanism. Disabling the knight's fork — the differential move of Equation 20 — costs a factor of 47 on the same average (a factor of 274 on Bent Cigar), significant on the same five problems; the fork is CA's principal rotation-invariant recombination device. The remaining ten mechanisms measure small individually: threefold repetition averages a ratio of 1.21 (significant on three problems, with mixed sign — protective on Bent Cigar and Rastrigin, marginally harmful on the composition function), the royal council is significantly *helpful* only on the welded beam (+2.9% when removed) while being significantly *harmful* to retain on two synthetic functions — consistent with its design purpose, the constraint-valley geometry of Equation 22, and suggesting a constraint-conditional gating as a refinement — and the other eight (sacrifice, pinning, windmill, opposition initialization, discovered attack, blockade, interference, castling) sit between 0.99 and 1.07 with at most one significant problem each. One methodological caveat applies to the small ratios: a one-at-a-time ablation measures each mechanism's marginal contribution in the presence of all others, and mechanisms with overlapping function can each appear individually dispensable even when they are not collectively so; the CA-static configuration, which disables the adaptive gating of all mechanisms simultaneously and is carried through every experiment in this paper, provides the complementary joint measurement.

The identity of the two load-bearing mechanisms is the most consequential finding in this paper's evaluation, because it connects Table 4 to the benchmark tables. En passant is the success-rule step-size family of Evolution Strategies; the fork is the differential-mutation family of Differential Evolution. The two algorithms that dominate every comparison in Section 4, Section 5, and Section 6 — CMA-ES and L-SHADE — are precisely the competition-refined embodiments of those two families, equipped with full adaptive machinery (covariance adaptation in one case, success-history parameter adaptation with population reduction in the other) where CA fires fixed-rate, fixed-shape versions embedded in its role architecture. The expectation formulated at the end of Section 2.12 is therefore confirmed rather than merely illustrated: CA's measured performance rests chiefly on its ES-like and DE-like components, and the specialized algorithms built purely on those components, with adaptation replacing fixed rates, outperform the composition that borrows them. This is simultaneously an honest account of why CA does not beat the modern state of the art and a mechanism-level validation that the architecture's active ingredients are real, identifiable, and correctly attributed.

7.2 Parameter sensitivity

A one-at-a-time sensitivity analysis perturbs each parameter group of Table 2 around the published default configuration — role fractions bracketed low and high, $\theta_0 \in \{0.05, 0.2\}$, castling period $c \in \{5, 20\}$, blockade threshold $\in \{15, 40\}$, and pinning cap $\in \{0.15, 0.50\}$ — on the same six problems and run protocol as Section 7.1. Table 22 reports each variant's mean-error ratio relative to the default configuration. The phase thresholds $\delta_{\text{open}}, \delta_{\text{end}}$ of Equation 17 are not re-swept here; the four-combination check reported in Section 2.11 stands.

Table 22: One-at-a-time parameter sensitivity: mean final-error ratio of each perturbed configuration relative to the default, averaged over six problems, with the number of problems deviating by more than twenty percent and the number of statistically significant deviations.

Parameter variant	Mean error ratio (variant / default)	$> 20\%$ deviation on (of 6)	Significant on (of 6)
Role fractions (.15/.20/.20/.25)	0.886	2	3
$\theta_0 = 0.2$	0.966	1	0
Castling period $c = 20$	0.978	2	0
Blockade threshold = 15	0.979	0	0
$\theta_0 = 0.05$	0.992	1	0
Pin cap = 0.15	0.993	0	0
Blockade threshold = 40	1.004	0	0
Pin cap = 0.50	1.007	0	0
Castling period $c = 5$	1.085	1	0
Role fractions (.05/.10/.10/.15)	13.410	3	5

The pattern is sharp: of sixty tested (variant, problem) cells, all eight statistically significant deviations belong to the role-fraction rows, and no other parameter produces a significant change on any problem, with mean ratios within ten percent of unity throughout. CA is therefore robust to its sacrifice constant, castling period, blockade threshold, and pinning cap at the granularity tested — a practitioner does not need to tune them — while the division of the population among piece roles is the one structural choice that matters. Its direction is informative in both senses. Shrinking the elite roles (more pawns) is harmful, catastrophically so on Bent Cigar (a factor of 73), where the elite ranks supply nearly all useful moves. Enlarging them is *beneficial* on several problems (mean ratio 0.89, significantly better on three): the published default of Table 2, fixed a priori and used unchanged for every result in this paper, is demonstrably not optimal everywhere, and this headroom is reported rather than exploited — no result elsewhere in the paper uses the improved fractions.

7.3 Computational cost

Cost is measured, not estimated: an exact counting wrapper intercepts every objective call, and wall-clock time is taken with a monotonic timer over ten runs per cell on an otherwise

idle machine, for all eight algorithms on six problems spanning the paper's sections (CEC-2017 F1, F13, F22 at $D = 30$; CEC-2022 F8 at $D = 20$; welded beam; signal timing P1 at its own $T = 300$ protocol). Table 23 reports mean seconds per run, mean evaluations consumed, the ratio of evaluations to the $N \times T$ core budget, and time relative to GA on the same problem.

Table 23: Measured wall-clock time and exact evaluation counts, all eight algorithms on six problems, ten runs per cell on an idle machine. "Evals / core budget" is the ratio of consumed evaluations to $N \times T$.

Problem	Algorithm	Mean time (s)	Mean evals	Evals / core budget	Time vs GA
F1	CA	0.312	16864	1.124	2.21
F1	CA-static	0.311	16761	1.117	2.20
F1	GWO	0.090	15030	1.002	0.64
F1	PSO	0.080	15030	1.002	0.57
F1	GA	0.141	15030	1.002	1.00
F1	WOA	0.555	15030	1.002	3.93
F1	L-SHADE	1.614	15500	1.033	11.43
F1	CMA-ES	1.065	15000	1.000	7.53
F13	CA	0.876	16710	1.114	1.42
F13	CA-static	0.816	16753	1.117	1.32
F13	GWO	0.540	15030	1.002	0.87
F13	PSO	0.529	15030	1.002	0.86
F13	GA	0.618	15030	1.002	1.00
F13	WOA	1.168	15030	1.002	1.89
F13	L-SHADE	2.078	15500	1.033	3.36
F13	CMA-ES	1.719	15000	1.000	2.78
F22	CA	1.700	16858	1.124	1.19
F22	CA-static	1.849	16758	1.117	1.29
F22	GWO	1.350	15030	1.002	0.94
F22	PSO	1.410	15030	1.002	0.99
F22	GA	1.430	15030	1.002	1.00
F22	WOA	1.945	15030	1.002	1.36
F22	L-SHADE	3.052	15500	1.033	2.13
F22	CMA-ES	2.352	15000	1.000	1.64
F8-2022-D20	CA	11.636	16765	1.118	1.12
F8-2022-D20	CA-static	11.795	16697	1.113	1.13
F8-2022-D20	GWO	12.051	15030	1.002	1.16
F8-2022-D20	PSO	10.023	15030	1.002	0.96
F8-2022-D20	GA	10.394	15030	1.002	1.00
F8-2022-D20	WOA	10.956	15030	1.002	1.05
F8-2022-D20	L-SHADE	11.554	15500	1.033	1.11
F8-2022-D20	CMA-ES	10.559	15000	1.000	1.02

Problem	Algorithm	Mean time (s)	Mean evals	Evals / core budget	Time vs GA
Welded-Beam	CA	0.298	16924	1.128	3.02
Welded-Beam	CA-static	0.296	16722	1.115	2.99
Welded-Beam	GWO	0.051	15030	1.002	0.52
Welded-Beam	PSO	0.038	15030	1.002	0.39
Welded-Beam	GA	0.099	15030	1.002	1.00
Welded-Beam	WOA	1.391	15030	1.002	14.07
Welded-Beam	L-SHADE	1.747	15500	1.033	17.66
Welded-Beam	CMA-ES	0.390	9000	0.600	3.95
SignalP1	CA	0.634	10205	1.134	2.75
SignalP1	CA-static	0.581	10043	1.116	2.53
SignalP1	GWO	0.193	9030	1.003	0.84
SignalP1	PSO	0.189	9030	1.003	0.82
SignalP1	GA	0.230	9030	1.003	1.00
SignalP1	WOA	5.797	9030	1.003	25.20
SignalP1	L-SHADE	6.714	9500	1.056	29.18
SignalP1	CMA-ES	0.746	9000	1.000	3.24

Three facts summarize the table. First, CA's evaluation overhead — the en-passant probes, castling probes, windmill extensions, and blockade responses documented throughout Section 2 — measures 12.4% over the core budget on average (range 11.4–13.4% across the six problems; CA-static 11.6%), which replaces the "ten to fifteen percent" estimate quoted in earlier sections with its measured value. Second, the budget asymmetry runs *against* the two strongest competitors: L-SHADE is capped at 3.3% over core and CMA-ES consumes at most exactly the core budget — on the welded beam it stops at sixty percent of it, having reached the numerical-precision floor of its stagnation tests — so the results of Section 4 through Section 6 cannot be attributed to evaluation-budget advantages held by the winners; the only algorithm holding a budget advantage in those tables is CA itself, and it is disclosed and measured. Third, wall-clock overhead depends on what dominates: on the cheapest objective CA costs about 2.2× GA's time per run, reflecting the control layer's per-iteration bookkeeping, while on the most expensive objective tested (CEC-2022 F8 at $D = 20$, where a single evaluation costs roughly forty times more than on Bent Cigar) all eight algorithms fall within about fifteen percent of one another — for expensive objectives, which are the practically interesting case, the choice among these algorithms is a choice about evaluations, not about control-layer arithmetic.

8 Transportation Application: Arterial Signal Coordination

8.1 Problem statement

Coordinated fixed-time signal timing along an urban arterial is a classic of transportation network optimization, and a highly deceptive and challenging one. The objective surface is multimodal, because different offset combinations produce different locally good "green waves." The variables are heterogeneous — a common cycle, per-intersection splits, offsets. And the delay model is nonlinear and, in practice, non-differentiable (Ceylan & Bell, 2004; Roess et al., 2019). In short, it is a natural first engineering test for CA.

Our test bed is an eight-intersection arterial with spacings of $450, 380, 520, 300, 610, 420$, and 350 m, a progression speed of 50 km/h, near-saturation two-way traffic (eastbound approach demands between roughly 1,240 and 1,400 veh/h), and conflicting cross-street demands at every junction (Figure 7). Each intersection runs a two-phase plan.

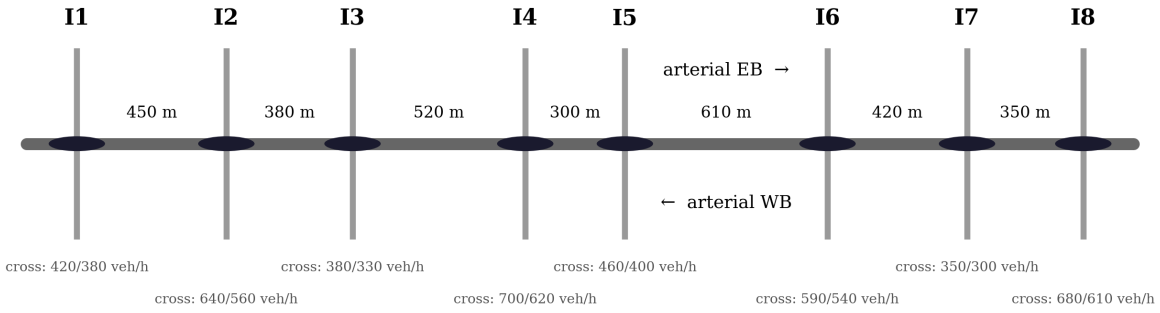

Figure 7: Layout of the eight-intersection arterial test bed.

8.2 Optimization model

The decision vector is

$$\mathbf{z} = (C,\ g_1, \dots, g_8,\ o_2, \dots, o_8) \in \mathbb{R}^{16}, \tag{26}$$

with a common cycle length $C \in [60, 140]$ s, arterial green splits $g_i \in [0.30, 0.75]$, and offsets o_i expressed as fractions of the cycle (intersection 1 serves as reference). The objective is total network delay in veh $\cdot$ h/h, built from three ingredients.

The first is uniform delay per approach, Webster's first term (Webster, 1958):

$$d_1 = \frac{C\,(1-g)^2}{2\,(1-\min(1,x)\,g)}, \tag{27}$$

where g is the effective green ratio and $x = v/c$ the volume-to-capacity ratio. The second is overflow delay from the HCM time-dependent formulation (Roess et al., 2019):

$$d_2 = 900\,T_a \left[(x-1) + \sqrt{(x-1)^2 + \frac{8\,k\,I\,x}{c\,T_a}} \right], \tag{28}$$

with analysis period $T_a = 0.25$ h, incremental delay factor $k = 0.5$, and upstream filtering factor $I = 1$. The third is a progression adjustment: platoons arriving from upstream are shifted by the offset difference minus the travel time, and the mismatch between platoon arrival and the local green window scales the uniform delay up or down, per direction. Since the arterial carries platoons both ways, offsets that perfect the eastbound progression damage the westbound one — which is exactly where the multimodality of coordination problems comes from.

Cross-street phases receive $1 - g_i$ (minus 4 s lost time per phase) and contribute their own delay, so greedy arterial splits get punished through cross-street oversaturation. All terms are summed over the sixteen arterial approaches and eight cross-street approaches.

8.3 Experimental setup

CA, CA-static (Section 2.11), GA, PSO, and GWO — the three strongest baselines from Section 3 plus CA and its ablation — run with $N = 30$, $T = 300$ iterations (9,000 core evaluations), and 30 independent runs each, again without any tuning. SA is excluded from this comparison given its uncompetitive performance in Section 3.

8.4 Results

Table 24 summarizes the final delays; Figure 8 and Figure 9 show the convergence behavior and the spread across runs.

Table 24: Final total delay over 30 runs (best mean in bold).

Algorithm	Mean delay (veh · h/h)	Std	Best	Worst
CA	169.963	4.871	166.525	179.149
CA-static	170.827	5.072	166.525	178.533
GA	171.160	4.895	166.526	178.470
PSO	**169.491**	3.666	166.526	179.946
GWO	170.506	2.818	166.739	177.987

Table 25: Wilcoxon rank-sum tests on the signal timing problem, CA versus each competitor.

Comparison	p-value	Result (= 0.05)
CA vs CA-static	6.249e-02	not significant
CA vs GA	2.002e-03	CA better
CA vs PSO	3.089e-02	CA worse
CA vs GWO	1.471e-02	CA better

Four key observations emerge from the results, highlighting both the strengths and limitations of the algorithms. First, all five configurations land within about one percent of each other in mean delay (169.5–171.2 veh · h/h), yet at this scale the differences are mostly resolvable: CA is significantly better than GA ($p = 0.002$) and GWO ($p = 0.015$), statistically tied with its own ablation CA-static ($p = 0.062$), and — the one result unfavorable to CA — significantly *worse* than PSO ($p = 0.031$), whose mean delay of 169.491 is the lowest in the table. Second, on the best-solution metric the ranking is reversed: CA and CA-static both reach 166.5254 veh · h/h, the lowest value found by any method, fractionally ahead of GA's and PSO's best

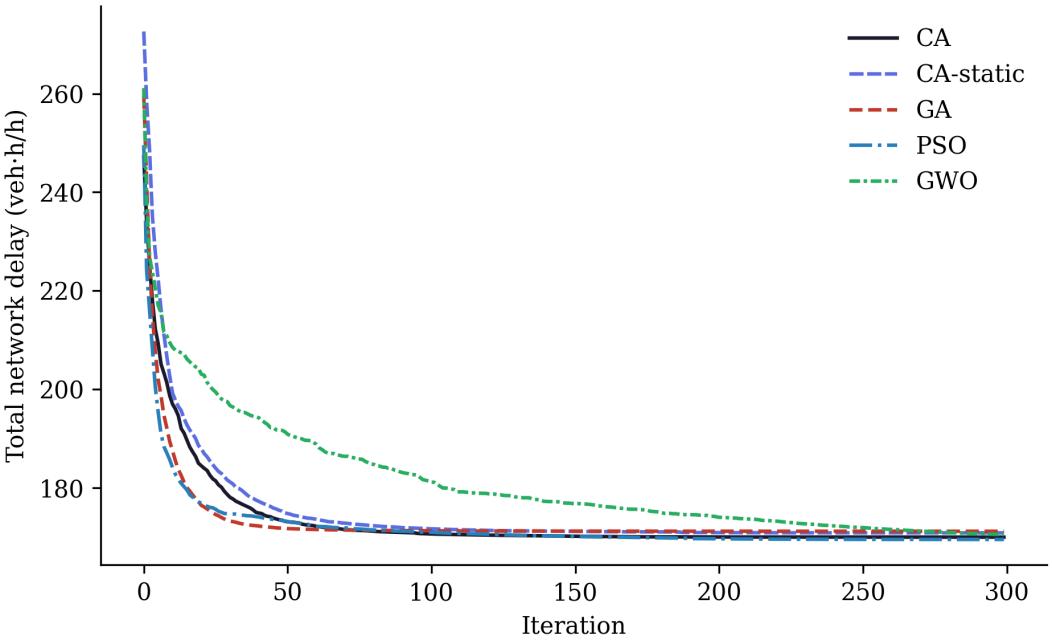

Figure 8: Median convergence on the arterial signal timing problem.

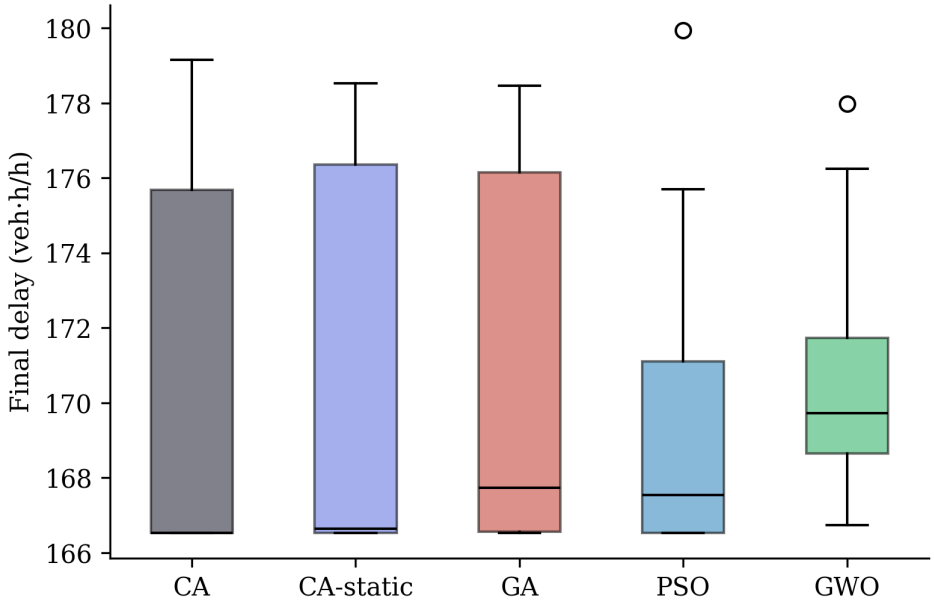

Figure 9: Distribution of final delays over 30 runs.

runs (166.526) and clearly ahead of GWO's (166.739) — so CA's operators are fully capable of locating the same high-quality coordination plan as the strongest baselines, even where its mean is lower than PSO's. Third, and the most notable pattern: the classical-suite ranking of Section 3, where GWO dominates, does not carry over here at all — GWO is the worst mean performer of the five. This constitutes a clear within-paper illustration of the No-Free-Lunch theorem (Wolpert & Macready, 1997). Fourth, the interpretation is kept modest and specific rather than characterized as an unqualified sweep: CA is competitive with, and on several pairwise comparisons better than, the state of the practice on this problem, but PSO's mean advantage is reported as such rather than minimized.

8.5 The best coordination plan found

The best plan discovered (by CA) selects the minimum cycle, $C = 60$ s — a sensible choice under Webster's guidance whenever splits can be kept efficient — with the timings of Table 26, drawn as a time–space diagram in Figure 10:

Table 26: Best signal timing plan found (CA).

Parameter	Value
Cycle length C	60.0 s
Arterial green split g1	0.633 (38.0 s)
Arterial green split g2	0.529 (31.7 s)
Arterial green split g3	0.667 (40.0 s)
Arterial green split g4	0.498 (29.9 s)
Arterial green split g5	0.624 (37.4 s)
Arterial green split g6	0.540 (32.4 s)
Arterial green split g7	0.678 (40.7 s)
Arterial green split g8	0.493 (29.6 s)
Offset o2	32.4 s
Offset o3	59.8 s
Offset o4	37.2 s
Offset o5	58.8 s
Offset o6	42.7 s
Offset o7	13.0 s
Offset o8	38.2 s
Total delay	**166.525 veh · h/h**

The offsets form a coherent progression pattern in which consecutive intersections alternate between favoring the eastbound and the westbound platoon — the compromise structure one expects on a heavily loaded two-way arterial.

8.6 Practical remarks

For a transportation agency, the practical implication is that CA can be applied to signal-coordination studies with the same confidence as GA, PSO, or GWO, and offers two features of practical relevance. The sacrifice mechanism provides a principled means of escaping poor offset basins without restarting the search, and the en-passant refinement improves the precision of splits and offsets late in the run without requiring a supplementary local search stage. Extension to cycle-by-cycle stochastic simulation objectives (VISSIM- or SUMO-in-the-loop) is

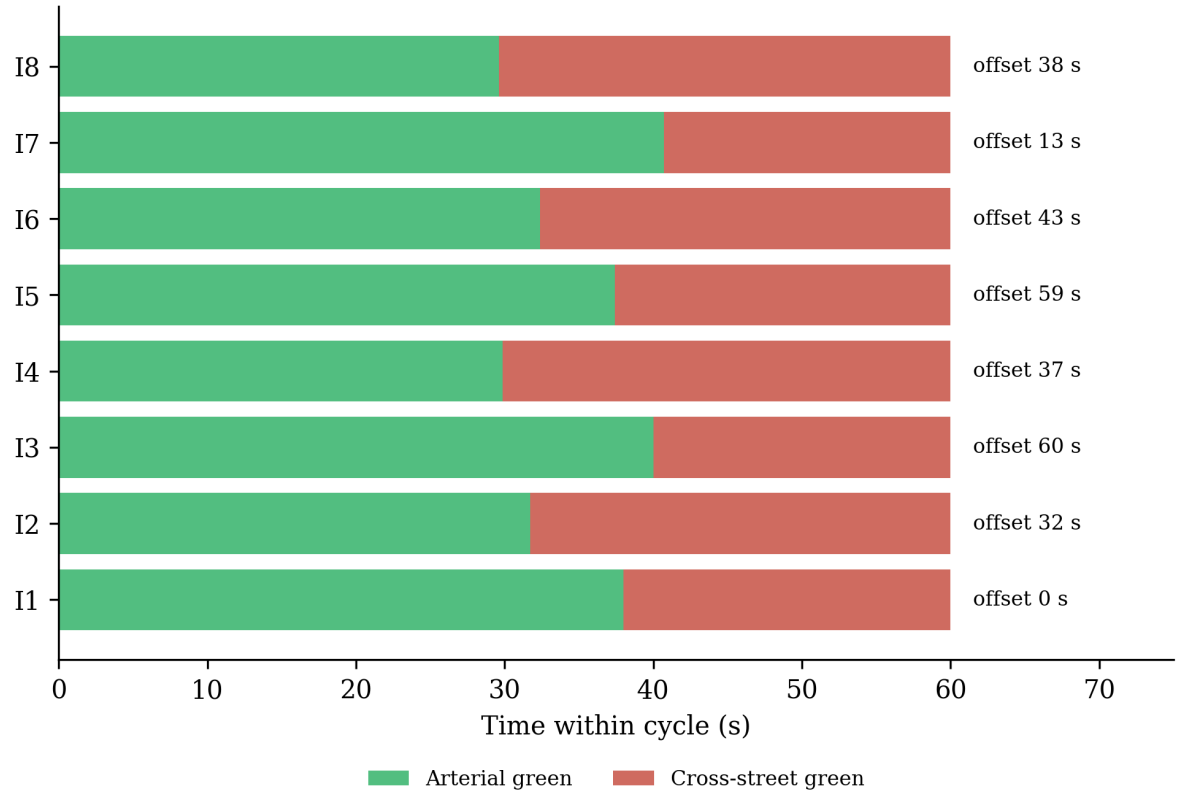

Figure 10: Time–space representation of the best plan found by CA.

straightforward, since CA requires only function values. The comparison against PSO merits explicit attention: on this particular sixteen-variable, periodic-offset landscape, PSO's velocity-based search exhibits a genuine, if small, advantage in mean delay, and an agency selecting between the two algorithms on this problem class alone should not assume CA will outperform PSO by default — the case for CA rests on the broader evidence presented in Section 4 and Section 6, rather than on this individual instance.

9 Extended Comparison on Independent Implementations

9.1 Motivation

Every baseline in Section 3 and Section 8 was implemented by the present authors, using the same code style and the same array-vectorized idiom as CA itself. That is standard practice, but it leaves open the question of whether CA remains competitive against implementations developed independently. To address this question, we employ `mealpy` (Van Thieu & Mirjalili, 2023), a large, actively maintained, third-party Python library that currently ships several hundred metaheuristic variants with a common `solve(problem, seed=...)` interface. Six well-known algorithms were evaluated exactly as `mealpy` ships them — without modification, using default hyperparameters — against CA on two transportation problems, one of which has a global optimum known in closed form.

The six `mealpy` competitors are the Whale Optimization Algorithm (WOA) (Mirjalili & Lewis, 2016), the Sine Cosine Algorithm (SCA) (Mirjalili, 2016b), the Ant Lion Optimizer (ALO) (Mirjalili, 2015b), Moth-Flame Optimization (MFO) (Mirjalili, 2015a), Harris Hawks

Optimization (HHO) (Heidari et al., 2019), and Differential Evolution (DE) (Storn & Price, 1997). The two competition-grade baselines of Section 4 — L-SHADE (via `niapy` (Vrbančič et al., 2018)) and CMA-ES (via `pycma` (Hansen et al., 2019)) — are also carried into both problems, so the transportation instances face the same strongest competitors as the synthetic suites. All methods search the identical unit hypercube — problem-specific bounds are folded into the decoding step rather than passed to the optimizers directly, so no algorithm gets an easier-to-search space than another. Every algorithm runs with population 30 for 300 iterations across 30 independent runs, with run r of every algorithm seeded identically at $\text{SEED}_0 + r$; the evaluation-budget accounting of Section 7.3 applies here in the same proportions (9,000 core evaluations; L-SHADE capped at 9,500).

9.2 Problem P1: the arterial signal timing instance

This is the same sixteen-variable coordination problem from Section 8. Table 27 reports the summary statistics; Figure 11 (left panel) shows median convergence.

Table 27: CA versus six mealpy algorithms and the two competition-grade baselines on the arterial signal timing problem (P1).

Algorithm	Mean	Std	Best	Worst	p-value (vs CA)	Result (= 0.05)
CA	169.963	4.871	166.525	179.149	—	—
WOA	199.357	21.475	172.715	281.873	7.762e-11	CA better
SCA	174.006	2.307	171.044	180.683	2.962e-03	CA better
ALO	173.684	5.201	167.413	187.575	1.086e-03	CA better
MFO	179.765	5.849	169.539	189.142	1.306e-07	CA better
HHO	192.375	18.304	173.106	260.020	5.317e-10	CA better
DE	196.222	11.382	173.619	222.264	9.445e-11	CA better
L-SHADE	167.802	3.191	166.525	176.446	3.093e-04	CA worse
CMA-ES	**167.151**	1.868	166.525	176.446	1.054e-02	CA worse

CA is significantly better than every one of the six `mealpy` competitors — including SCA and ALO, its closest rivals in that group, at $p = 0.003$ and $p = 0.001$ respectively. WOA, HHO, and DE exhibit substantially lower performance and markedly higher run-to-run variance (standard deviations of 11–21 veh · h/h, versus 4.9 for CA), which suggests they struggle more than CA does to reliably re-find a good coordination plan across the sixteen-dimensional, periodic offset space. The two competition-grade baselines invert the comparison, as they do everywhere in this paper: L-SHADE (167.80) and CMA-ES (167.15) both hold significantly better mean delays than CA (169.96), by margins of 1.3 to 1.7 percent, and all three methods reach the same best-of-thirty-runs plan at 166.525 veh · h/h. The scope of these results should be stated precisely: GA and PSO, our own implementations from Section 8, achieve comparable

performance to CA on this problem (Table 24) — so the relevant conclusion is not that CA is uniquely strong on signal timing, but that its standing among widely used algorithms holds against implementations by independent authors, while the specialized adaptive methods retain a small, statistically resolvable advantage in mean delay.

9.3 Problem P2: continuous berth allocation with a known optimum

Independent implementations address one concern; a problem with a *known* optimal value addresses another, namely whether an algorithm's apparent superiority is an artifact of comparing relative suboptimality on a problem that none of the algorithms under comparison can solve. We borrow Example 1.9 of Teodorović & Janić (2020) (Chapter 1): fifteen ships of known length, arrival time, and required service time must be assigned mooring times and berth positions along a continuous 1,000 m quay within a 1,920-minute horizon, so as to minimize total time spent in port. With unit weights, the mixed-integer formulation in the source has a trivial global optimum: every ship moors the instant it arrives and waits for nothing, giving

$$F^{*} = \sum_{i=1}^{15} p_i = 4{,}860 \text{ min}.$$

Achieving that bound requires the ship-pair schedule to be simultaneously conflict-free along both the time axis and the quay-position axis — a combinatorial matching problem embedded within a continuous one, which is what makes the instance a substantive test for a population-based search rather than a superficial one. We encode each ship's mooring time $u_i \in [a_i,\ T - p_i]$ and berth position $v_i \in [0,\ S - s_i]$ as decision variables and penalize, rather than hard-constrain, the pairwise rectangle overlap in the time–space diagram, so the search can pass through momentarily infeasible plans on its way to a good one — the same soft-constraint philosophy already used for cross-street oversaturation in Section 8.

Table 28: CA versus six mealpy algorithms and the two competition-grade baselines on the continuous berth allocation problem (P2), objective in minutes.

Algorithm	Mean	Std	Best	Worst	p-value (vs CA)	Result (= 0.05)
CA	7934.2	973.8	6277.9	10257.8	—	—
WOA	45507.4	21345.8	9388.5	93564.7	3.879e-11	CA better
SCA	34567.6	9683.6	19217.2	57890.7	2.872e-11	CA better
ALO	9912.0	3544.1	7345.0	27033.0	2.320e-04	CA better
MFO	21914.8	12102.4	9112.4	52125.0	4.734e-11	CA better
HHO	48415.6	25410.2	12102.0	133361.1	2.872e-11	CA better
DE	25765.7	12631.0	9768.9	54967.4	3.879e-11	CA better
L-SHADE	**6269.1**	688.6	5283.7	8712.7	1.256e-08	CA worse
CMA-ES	6521.8	537.5	5424.7	7620.9	1.204e-07	CA worse

Against the `mealpy` group, CA's advantage increases substantially on this problem: its mean objective is roughly a sixth of WOA's or HHO's, and it is significantly better than every one of the six, ALO included ($p = 2.3 \times 10^{-4}$), which is otherwise its nearest rival in that group by a considerable margin. Figure 11 (right panel) illustrates the pattern — among the seven methods shown, CA and ALO are the only two that make sustained progress past iteration 50, while WOA, SCA, MFO, HHO, and DE largely plateau in the first fifty iterations. The two competition-grade baselines again lead: L-SHADE's mean of 6,269 and CMA-ES's of 6,522 are both significantly better than CA's 7,934, this time by the widest relative margins observed on any problem in this paper outside the synthetic suites (18–21% in mean objective).

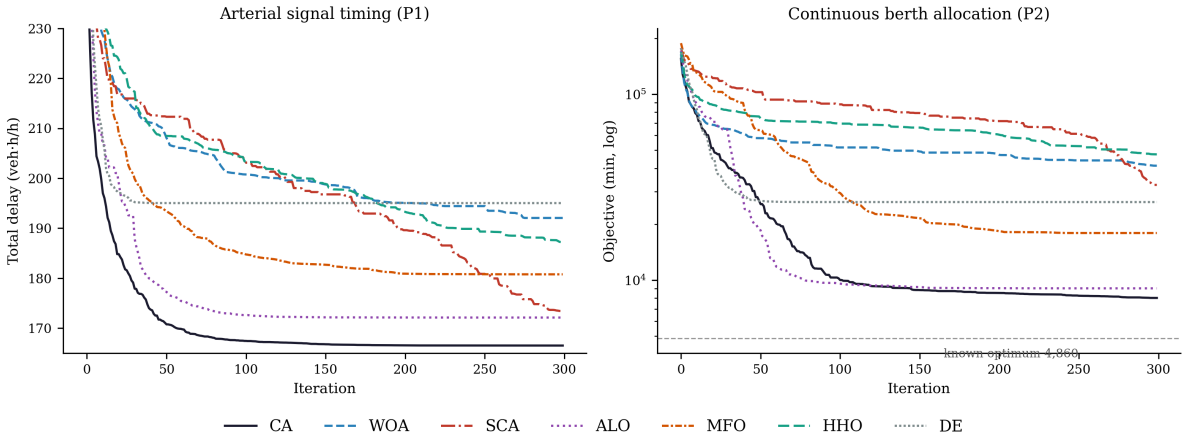

Figure 11: Median convergence on the two transportation problems, CA versus six mealpy algorithms (30 runs). The berth-allocation panel marks the known global optimum.

The distance of even the best runs from $F^* = 4{,}860$ should be stated explicitly. Table 29 reports the optimality gap of each algorithm's best run, together with the residual overlap of its decoded plan.

Table 29: Optimality gap and feasibility of the best plan found by each algorithm on P2.

Algorithm	Best objective (min)	Gap to optimum	Overlap of best plan (min · m)
CA	6277.9	29.17%	0.0
WOA	9388.5	93.18%	878.9
SCA	19217.2	295.42%	11752.0
ALO	7345.0	51.13%	0.0
MFO	9112.4	87.50%	1873.6
HHO	12102.0	149.01%	1963.4
DE	9768.9	101.01%	0.0
L-SHADE	5283.7	8.72%	0.5
CMA-ES	5424.7	11.62%	0.0

CA's best run sits 29% above the true optimum — a substantial gap, which we explicitly acknowledge without obfuscation. Three aspects of the table warrant separate discussion. First, the smallest gaps belong to the competition-grade methods: L-SHADE reaches 8.7% (with a numerically negligible residual overlap of 0.5 min · m in its best plan) and CMA-ES 11.6% on a fully feasible plan — the closest any general-purpose method in this study comes to the combinatorial optimum, and clear evidence that the instance rewards stronger adaptive

machinery rather than being uniformly hard. Second, among the seven remaining methods CA's gap is the smallest by a considerable margin — 22 to 266 percentage points closer to F^* than the six `mealpy` competitors, including a 22-point margin over ALO, its nearest rival in that group. Third, the overlap column shows that CA's best plan is fully feasible — ships do not actually collide in the time–space diagram — which is not true of WOA, SCA, MFO, or HHO, whose reported objectives still include unresolved penalty mass from residual overlaps; ALO and DE also land on fully feasible plans, but at substantially larger gaps. Figure 12 shows CA's best schedule; every ship occupies its own rectangle with no overlap, though several ships wait appreciably longer than their arrival time, which accounts for the remaining 29% of the gap.

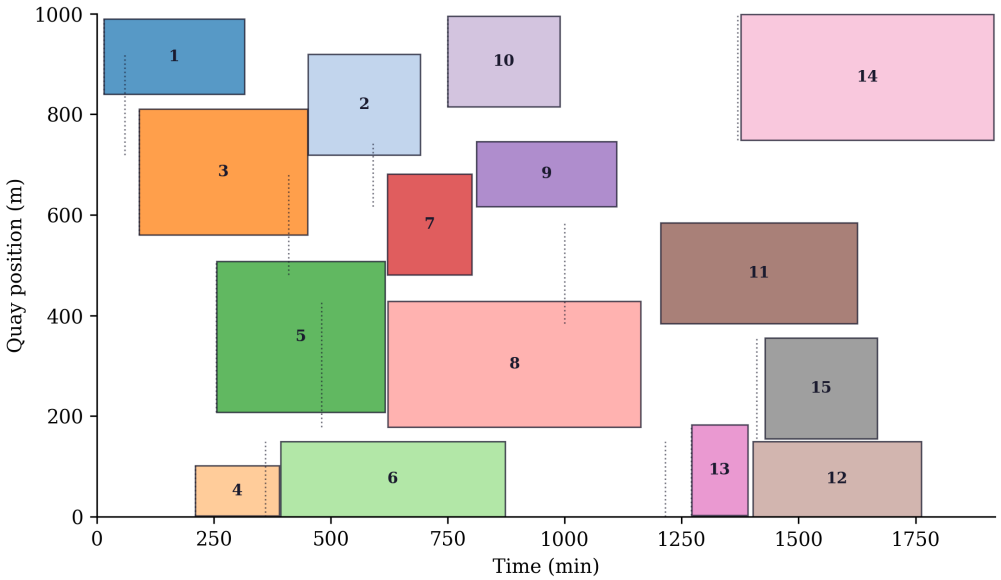

Figure 12: Time–space diagram of the best berth allocation plan found by CA. Dotted lines mark each ship's arrival time; the gap between the arrival line and the left edge of a ship's rectangle is its waiting time.

9.4 Scope of the empirical claims

The scope of this comparison should be stated precisely. It demonstrates that CA's competitiveness on transportation problems is not an artifact of implementation quality on the baseline side, since all eight competitors in this section are third-party code that was not modified. It does not demonstrate that CA is close to solving berth allocation to global optimality — a 29% gap remains, against 9–12% for the two competition-grade methods — and the problem clearly warrants algorithms better tuned to its combinatorial structure than any general-purpose metaheuristic evaluated here. The conclusions drawn from this section are two, and they mirror the rest of the paper: among the seven widely used general-purpose metaheuristics run with default settings (CA and the six `mealpy` algorithms), CA reaches the best and most consistently feasible solutions on both transportation instances tested; and L-SHADE and CMA-ES outperform CA on both instances, by small margins on signal timing and by decisive ones on berth allocation.

10 Conclusions and Future Work

10.1 Summary of findings

This paper introduced the Chess Algorithm (CA), a population-based metaheuristic whose defining feature is a heterogeneous-role architecture: agents dynamically assume the roles of Queens, Rooks, Bishops, Knights, and Pawns around an elite King, each with its own movement geometry, while strategic mechanisms from chess theory — development, sacrifice, pinning, castling, en passant, and threefold repetition — govern the search throughout, and a state machine reads the divergence, initiative, and local-search success of the population each iteration to decide which of a richer tactical set (Novotny interference, the knight's fork, the royal council, and a blockade response combining a King's march with a Tal-style speculative sacrifice) applies at any given moment (Section 2). Every mechanism is mapped onto its established operator family (Section 2.12), directly engaging the metaphor critique of Sörensen (2015) rather than dismissing it. An ablation configuration, CA-static, disables the adaptive layer and is carried through every experiment as a control.

The principal findings are summarized as follows. On six classical 30-dimensional benchmarks under matched budgets, CA significantly outperforms PSO and SA on every function and GA on five of six, while GWO remains significantly stronger on that classical suite — and CA-static is at least as good as CA here, indicating that the adaptive layer's overhead is not without cost on smooth, low-structure landscapes (Section 3). On the full CEC-2017 suite — all 29 usable functions, validated through a two-library data-integrity audit that identified six defective implementations in the primary third-party port and restored all six from an independent, official-data source — CA achieves the best mean Friedman rank among the classical/moderate roster (CA, CA-static, GWO, PSO, GA, WOA), losing one comparison of 29 to each of GWO and PSO and none to WOA, while L-SHADE and CMA-ES rank first and second overall and beat CA on the large majority of functions (Section 4). The same ordering holds on CEC-2022 at both official dimensionalities, sixteen (function, dimension) cells validated by an analogous audit that additionally excluded a non-discriminative function under a mechanical, pre-registered rule (Section 5). On seven classic constrained engineering design problems, CA is again the best of the classical/moderate roster, losing exactly once across 35 comparisons to that group, while L-SHADE and CMA-ES beat it on the large majority of problems, in several cases reaching literature-optimum values with near-zero run-to-run variance (Section 6). A granular ablation of all twelve strategic mechanisms identifies exactly two — en passant and the knight's fork — as carrying the large majority of CA's measured performance, and connects this finding to the ranking above: those two mechanisms are the fixed-rate analogues of the success-rule step-adaptation and differential-mutation families that CMA-ES and L-SHADE embody in fully adaptive form, so the algorithms that win are the specialized versions of CA's own load-bearing components. A parameter sensitivity analysis finds CA robust to all tested parameters except the role fractions, and a measured cost analysis replaces an earlier estimate of CA's evaluation overhead with a precise figure of 12.4% (Section 7). On the eight-intersection arterial signal-timing problem, CA is significantly better than GA and GWO, statistically tied with CA-static, significantly worse in mean delay than PSO, and narrowly but significantly worse than L-SHADE and CMA-ES, even though CA and CA-static both reach the best individual solution found by any method (Section 8). Against six third-party implementations from the `mealpy` library plus L-SHADE and CMA-ES, CA is significantly better than all six `mealpy` baselines on both transportation instances tested, including a continuous berth allocation problem with a known optimum, where it also produces a fully feasible best-found plan; L-SHADE and CMA-ES outperform CA on both instances, decisively on berth allocation (Section 9).

Three specific limitations of CA were identified during the evaluation, all of which are objectively reported and two of which are now mechanistically explained rather than merely observed. CA does not match the two competition-grade optimizers, L-SHADE and CMA-ES, on any problem class tested in this paper; Section 7.1 traces this directly to CA's fixed-rate, fixed-shape implementation of the two operator families those methods specialize in and adapt fully. Against GA on CEC-2017, CA wins 10, ties 9, and loses 10 of 29 functions, and the ten losses cluster tightly on Rastrigin-family, hybrid, and composition landscapes whose basins are separated by distances comparable to the search domain itself — precisely where GA's BLX-α crossover, which can relocate a candidate solution across the entire domain in one step, has a structural advantage over every CA operator, each of which is anchored to the King, an elite peer, or a bounded local neighborhood. And against PSO on the arterial signal-timing problem, CA's mean delay is significantly worse, even though its best-found plan is not. All three outcomes are interpreted as manifestations of the No-Free-Lunch theorem (Wolpert & Macready, 1997) rather than as deficiencies to be minimized; the GA trade-off is addressed further in the future-work discussion below.

10.2 Limitations

The boundaries of the evidence should be stated as explicitly as the results. Scalability beyond $D = 30$ (CEC-2017/engineering) and $D = 20$ (CEC-2022) is untested; studies at 50–100 dimensions remain future work. CA's measured evaluation overhead is 12.4% over the shared core budget (Section 7.3) — small relative to the order-of-magnitude gaps reported against the classical roster, but a real cost, and one that does not favor CA relative to L-SHADE (3.3% over core) or CMA-ES (at most exactly the core budget). The transportation studies use analytic delay and penalty models rather than microsimulation. The granular ablation and parameter sensitivity analyses of Section 7 cover six representative problems rather than the full benchmark set, for computational tractability; the phase thresholds of Equation 17 were checked on two functions rather than swept exhaustively. The opposition-based initialization of Section 2.11 showed a mixed effect in early tuning and a small positive effect in the full ablation (ratio 1.03, significant on one of six problems) — a modest, not dramatic, case for keeping it. And the CA vs. GA comparison of Section 4 identifies, but does not yet close, a specific class of landscape — long-range, unstructured, multimodal — where CA's currently local operator vocabulary is under-powered relative to GA's macro-scale crossover.

10.3 Future work

Several directions strike us as natural, and two are now considerably more concrete than the others. The CA vs. GA result motivates a chess-native answer to long-range basin-hopping: a *pawn promotion* operator, in which an agent that survives several consecutive blockade events (Section 2.11) without producing a King improvement is, for one iteration, promoted from the bounded local geometry of Section 2.11 to a BLX-α-style macro-crossover against a distant population member — a discontinuous, large-scale change in capability, exactly as a pawn reaching the eighth rank is not incrementally improved but instantly becomes the most powerful piece on the board. The ablation of Section 7.1 motivates a second, more direct line: since en passant and the knight's fork are already CA's load-bearing operators, replacing their fixed adaptation rates with the success-history mechanism of L-SHADE or the covariance adaptation of CMA-ES is now a specific, testable hypothesis about closing the gap to those methods, rather than a speculative one — CA's role architecture and phase-conditioned gating would remain, but the two central operators would inherit the adaptation machinery of the algorithms that currently outperform them. A third, chess-native alternative to both, drawn from the variant

Bughouse/Crazyhouse, would "drop" a coordinate subset of one agent onto an unrelated one, a cross-dimensional recombination distinct from the segment-local Novotny interference already in CA. None of the three is implemented or benchmarked here; each is a natural next step precisely because this paper's evidence identifies, rather than merely speculates about, the mechanism and the landscape class it should target.

Beyond this, binary, discrete, and multi-objective variants of CA constitute a natural further direction — a Pareto "King's court" of nondominated solutions is a close conceptual fit for the role architecture. On the theoretical side, the en-passant success rule warrants analysis as a $(1+\lambda)$-ES embedded within a population method, a connection Section 2.12 makes explicit but does not develop formally. On the applications side, network-wide signal optimization, transit network design, and simulation-based optimization (Osorio & Bierlaire, 2013) are all accessible through the same interface, and a systematic restoration of the CEC-2017 hybrid/composition landscape class at $D = 50$ and $D = 100$ would test whether the GA trade-off persists at higher dimension.

10.4 Concluding remark

The value of a new metaheuristic should be assessed not by its performance on any single benchmark table but by whether its underlying architectural principles generalize across problem classes, and whether the claims made for it survive contact with the strongest available competitors. The evidence assembled here — six classical functions, all 29 usable CEC-2017 functions, sixteen validated CEC-2022 cells, seven engineering design problems, two transportation applications, a twelve-mechanism ablation, a parameter sensitivity analysis, and a measured cost analysis — supports a claim narrower than the one a first draft of this work might have made. CA's heterogeneous-role architecture, together with the adaptive layer's capacity to select context-appropriate operators, is a transparent, mathematically specified, and consistently competitive basis for global optimization against classical and widely used metaheuristics, including on the transportation-network problems that motivated this work; it is not competitive with the specialized, competition-grade optimizers built on the same operator families in fully adaptive form, and the ablation study identifies exactly which components explain both halves of that statement. Stating the second half without qualification is what the audit and ablation machinery of this paper is for.

Data and code availability

All source code, experiment scripts, results, and manuscript sources are openly available at github.com/sabernaseralavi-60/2026_Chess-Algorithm; the rendered article is served at sabernaseralavi-60.github.io/2026_Chess-Algorithm, with the PDF at `paper.pdf`. A Persian translation (`paper-fa.qmd`) is included in the repository and can be rendered locally to Word with Quarto. Every figure and table in this paper is generated by committed scripts:

```
python src/phase_threshold_check.py     # phase-transition threshold check (@sec-adaptive)
python src/run_benchmarks.py            # six-function benchmark experiments (@sec-benchmar
python src/traffic_case_study.py        # arterial signal timing case study (@sec-transport
python src/mealpy_comparison.py         # extended third-party comparison (@sec-mealpy)
python src/cec2017_full_run.py --part i/4   # full CEC-2017 suite, i = 0..3 (@sec-cec2017)
python src/cec2022_full_run.py --part i/4   # full CEC-2022 suite, i = 0..3 (@sec-cec2022)
python src/sota_addon_run.py            # L-SHADE + CMA-ES on the existing suites
```

```
python src/engineering_full_run.py      # 7 engineering design problems (@sec-engineering)
python src/cec2017_restoration_audit.py # cross-library audit of excluded labels (@sec-opfu
python src/cec2017_restore_run.py       # 8-algorithm rerun of the restored labels
python src/ablation_study.py            # granular per-mechanism ablation (@sec-ablation)
python src/sensitivity_study.py         # OAT parameter sensitivity (@sec-sensitivity)
python src/timing_study.py              # measured wall-clock + evaluation counts (@sec-cos
python src/phase2_3_analysis.py         # CEC-2017/engineering merge, audit, Friedman/Wilco
python src/cec2022_analysis.py          # CEC-2022 merge, audit, ranks, figure
python src/generate_markdown_tables.py  # this paper's result tables
python src/generate_timing_table.py     # the measured-cost table (@sec-cost)
python src/generate_latex_tables.py     # camera-ready LaTeX tables (results/latex_tables_f
quarto render                           # build HTML + PDF + Persian docx
```

In `src/algorithms.py`, CA is the function `chess_algorithm_v3` and its ablation CA-static is `chess_algorithm_v2`; the module also retains an earlier non-adaptive prototype (`chess_algorithm`) from the algorithm's development history, kept for archival reference but not used to produce any result in this paper. The single-mechanism ablation switches of Section 7.1 are keyword arguments of `chess_algorithm_v3` whose defaults reproduce the published behavior bit-for-bit (verified against the pre-change implementation on identical seeds before any ablation run). Random seeds are fixed in every script; the results in this paper correspond exactly to the committed outputs in `results/` and `figures/`. The data-integrity audits of Section 4.2 and Section 5 are likewise reproducible rather than asserted: `results/cec2017_opfunu_defect_evidence.csv` preserves the defective `opfunu` raw statistics for the six excluded CEC-2017 labels, and `results/cec2022_stats_raw_unaudited.csv` retains the full pre-audit CEC-2022 numbers, so a reader can re-run the same checks against a newer `opfunu` release rather than take our exclusion lists on faith. Readers who want the detailed per-function and per-problem tables in camera-ready LaTeX rather than the paper's own tables may use `results/latex_tables_final.tex` directly.

About the corresponding author

Seyed Saber Naseralavi is an Assistant Professor in the Department of Civil Engineering at Shahid Bahonar University of Kerman, Iran. He received his B.Sc. in Civil Engineering from Shahid Bahonar University of Kerman (2004), his M.Sc. in Highway and Transportation Engineering from Iran University of Science and Technology (2006), and his Ph.D. in Highway and Transportation Engineering from Tarbiat Modares University (2011), graduating first in his doctoral cohort at the Faculty of Civil and Environmental Engineering. His research spans traffic safety modeling and surrogate safety measures, traffic flow theory and simulation, travel behavior, machine learning applications in transportation, and metaheuristic optimization for transportation network design. He has authored over 50 refereed journal articles and 80 conference papers, supervised numerous graduate theses, and reviews for several international journals. A provincial chess champion in his youth — first place in the under-12 and under-14 tournaments of Kerman province — he brings to this work a lifelong familiarity with the strategic structure of the game that inspired the algorithm.

References

Awad, N. H., Ali, M. Z., Liang, J. J., Qu, B. Y., & Suganthan, P. N. (2016). *Problem definitions and evaluation criteria for the CEC 2017 special session and competition on single objective real-parameter numerical optimization* [Technical Report]. Nanyang Technological University.

Beyer, H.-G., & Schwefel, H.-P. (2002). Evolution strategies — a comprehensive introduction. *Natural Computing*, *1*(1), 3–52. https://doi.org/10.1023/A:1015059928466

Ceylan, H., & Bell, M. G. H. (2004). Traffic signal timing optimisation based on genetic algorithm approach, including drivers' routing. *Transportation Research Part B: Methodological*, *38*(4), 329–342. https://doi.org/10.1016/S0191-2615(03)00015-8

Derrac, J., García, S., Molina, D., & Herrera, F. (2011). A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm and Evolutionary Computation*, *1*(1), 3–18. https://doi.org/10.1016/j.swevo.2011.02.002

Dorigo, M., Maniezzo, V., & Colorni, A. (1996). Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, *26*(1), 29–41. https://doi.org/10.1109/3477.484436

Eiben, Á. E., Hinterding, R., & Michalewicz, Z. (1999). Parameter control in evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, *3*(2), 124–141. https://doi.org/10.1109/4235.771166

Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.

Hansen, N., Akimoto, Y., & Baudis, P. (2019). *CMA-ES/pycma on GitHub*. Zenodo. https://doi.org/10.5281/zenodo.2559634

Hansen, N., & Ostermeier, A. (2001). Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, *9*(2), 159–195. https://doi.org/10.1162/106365601750190398

Heidari, A. A., Mirjalili, S., Faris, H., Aljarah, I., Mafarja, M., & Chen, H. (2019). Harris hawks optimization: Algorithm and applications. *Future Generation Computer Systems*, *97*, 849–872. https://doi.org/10.1016/j.future.2019.02.028

Holland, J. H. (1975). *Adaptation in natural and artificial systems*. University of Michigan Press.

Karaboga, D., & Basturk, B. (2007). A powerful and efficient algorithm for numerical function optimization: Artificial bee colony (ABC) algorithm. *Journal of Global Optimization*, *39*(3), 459–471. https://doi.org/10.1007/s10898-007-9149-x

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of ICNN'95 — International Conference on Neural Networks*, *4*, 1942–1948. https://doi.org/10.1109/ICNN.1995.488968

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, *220*(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

Kumar, A., Price, K. V., Mohamed, A. W., Hadi, A. A., & Suganthan, P. N. (2021). *Problem definitions and evaluation criteria for the CEC 2022 special session and competition on single objective bound constrained numerical optimization* [Technical Report]. Nanyang Technological University.

Mantegna, R. N. (1994). Fast, accurate algorithm for numerical simulation of Lévy stable stochastic processes. *Physical Review E*, *49*(5), 4677–4683. https://doi.org/10.1103/PhysRevE.49.4677

Mirjalili, S. (2015a). Moth-flame optimization algorithm: A novel nature-inspired heuristic paradigm. *Knowledge-Based Systems*, *89*, 228–249. https://doi.org/10.1016/j.knosys.2015.07.006

Mirjalili, S. (2015b). The ant lion optimizer. *Advances in Engineering Software*, *83*, 80–98.

https://doi.org/10.1016/j.advengsoft.2015.01.010
Mirjalili, S. (2016a). Dragonfly algorithm: A new meta-heuristic optimization technique for solving single-objective, discrete, and multi-objective problems. *Neural Computing and Applications*, *27*(4), 1053–1073. https://doi.org/10.1007/s00521-015-1920-1
Mirjalili, S. (2016b). SCA: A sine cosine algorithm for solving optimization problems. *Knowledge-Based Systems*, *96*, 120–133. https://doi.org/10.1016/j.knosys.2015.12.022
Mirjalili, S., Gandomi, A. H., Mirjalili, S. Z., Saremi, S., Faris, H., & Mirjalili, S. M. (2017). Salp swarm algorithm: A bio-inspired optimizer for engineering design problems. *Advances in Engineering Software*, *114*, 163–191. https://doi.org/10.1016/j.advengsoft.2017.07.002
Mirjalili, S., & Lewis, A. (2016). The whale optimization algorithm. *Advances in Engineering Software*, *95*, 51–67. https://doi.org/10.1016/j.advengsoft.2016.01.008
Mirjalili, S., Mirjalili, S. M., & Lewis, A. (2014). Grey wolf optimizer. *Advances in Engineering Software*, *69*, 46–61. https://doi.org/10.1016/j.advengsoft.2013.12.007
Nelder, J. A., & Mead, R. (1965). A simplex method for function minimization. *The Computer Journal*, *7*(4), 308–313. https://doi.org/10.1093/comjnl/7.4.308
Osorio, C., & Bierlaire, M. (2013). A simulation-based optimization framework for urban transportation problems. *Operations Research*, *61*(6), 1333–1345. https://doi.org/10.1287/opre.2013.1226
Roess, R. P., Prassas, E. S., & McShane, W. R. (2019). *Traffic engineering* (5th ed.). Pearson.
Sörensen, K. (2015). Metaheuristics—the metaphor exposed. *International Transactions in Operational Research*, *22*(1), 3–18. https://doi.org/10.1111/itor.12001
Storn, R., & Price, K. (1997). Differential evolution — a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, *11*(4), 341–359. https://doi.org/10.1023/A:1008202821328
Tanabe, R., & Fukunaga, A. S. (2014). Improving the search performance of SHADE using linear population size reduction. *2014 IEEE Congress on Evolutionary Computation (CEC)*, 1658–1665. https://doi.org/10.1109/CEC.2014.6900380
Teodorović, D., & Janić, M. (2020). *Quantitative methods in transportation*. CRC Press. https://doi.org/10.1201/9780429316449
Tilley, D. (2020). *cec2017-py: Python implementation of the CEC 2017 benchmark functions*. https://github.com/tilleyd/cec2017-py.
Tizhoosh, H. R. (2005). Opposition-based learning: A new scheme for machine intelligence. *International Conference on Computational Intelligence for Modelling, Control and Automation (CIMCA 2005)*, *1*, 695–701. https://doi.org/10.1109/CIMCA.2005.1631345
Van Thieu, N., & Mirjalili, S. (2023). MEALPY: An open-source library for latest meta-heuristic algorithms in Python. *Journal of Systems Architecture*, *139*, 102871. https://doi.org/10.1016/j.sysarc.2023.102871
Vrbančič, G., Brezočnik, L., Mlakar, U., Fister, D., & Fister Jr., I. (2018). NiaPy: Python microframework for building nature-inspired algorithms. *Journal of Open Source Software*, *3*(23), 613. https://doi.org/10.21105/joss.00613
Webster, F. V. (1958). *Traffic signal settings* (Road Research Technical Paper No. 39). Road Research Laboratory.
Wolpert, D. H., & Macready, W. G. (1997). No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, *1*(1), 67–82. https://doi.org/10.1109/4235.585893
Yang, X.-S., & Deb, S. (2009). Cuckoo search via Lévy flights. *2009 World Congress on Nature & Biologically Inspired Computing (NaBIC)*, 210–214. https://doi.org/10.1109/NABIC.2009.5393690